

Python によるデータ解析と高速化

Ichiro KOMAE

2026 年 4 月 9 日

目次

はじめに	9
第 1 章 このテキストの使い方	11
1.1 誰のためのテキストか	11
1.2 何を順に学ぶのか	11
1.3 本書で大切にしたい姿勢	12
1.4 章の読み進め方	12
1.5 併用するとよい公開資料	12
1.6 この本の後半で扱う実践的な内容	13
第 2 章 シェルとファイル操作	15
2.1 ターミナルとシェル	15
2.2 Windows では WSL を使う	15
2.3 コマンド、オプション、引数	15
2.4 困ったときは説明を読む	16
2.5 今どこにいるかを確認する	16
2.6 移動する	17
2.7 パスを読む	17
2.8 ディレクトリを作る	18
2.9 ファイルの中身を見る	18
2.10 コピーする: cp	18
2.11 移動する、名前を変える: mv	19
2.11.1 改名の例	19
2.11.2 移動の例	20
2.12 削除する: rm	20
2.13 環境変数を使う	20
2.14 PATH と実行ファイル	21
2.15 権限を読む	21
2.16 スクリプトを直接実行する	22
第 3 章 Python 環境を整える	23
3.1 何を分けて考えるべきか	23
3.2 今の Python を確認する	23
3.3 pyenv で複数版の Python を管理する	23
3.3.1 macOS での準備	23

3.3.2	WSL / Ubuntu での準備	24
3.3.3	pyenv を入れる	24
3.3.4	どの設定ファイルを編集するか決める	24
3.3.5	pyenv をシェルへ組み込む	24
3.3.6	版を入れて切り替える	25
3.4	venv でプロジェクトごとの仮想環境を作る	25
3.4.1	pip と python -m pip	26
3.5	JupyterLab を併用する	26
3.6	uv はプロジェクト管理に使う	27
3.6.1	まずは既定の uv init を使う	27
3.6.2	必要ならオプションを使う	27
3.6.3	依存関係を追加する	28
3.6.4	環境をそろえる	28
3.6.5	コマンドを実行する	28
3.6.6	Python 版の固定にも使える	28
3.7	標準ライブラリと外部ライブラリ	29
3.8	解析用のライブラリを入れる	29
3.9	コマンドライン引数と設定ファイルを使い分ける	29
3.9.1	設定ファイルを読む	30
3.9.2	マジックナンバーを避ける	31
第 4 章	Python の基本	33
4.1	変数と基本型	33
4.2	数値と演算子	33
4.3	文字列	34
4.4	リスト、タプル、辞書	34
4.5	リストのメソッド	34
4.6	インデックスとスライス	35
4.7	条件分岐	35
4.8	反復処理	36
4.9	range と while	36
4.10	break, continue, else	37
4.11	内包表記	37
4.12	関数を分ける理由	37
4.13	関数の引数	38
4.14	モジュールと import	38
4.15	自作モジュールへ分ける	39
4.16	スクリプトとして実行する	39

4.17	エラーは失敗ではなく情報である	40
4.18	docstring とヘルプ	40
4.19	名前付けと PEP 8	40
4.20	マジックナンバーを避ける	41
4.21	例外処理	41
第 5 章	入出力とファイル	43
5.1	open と with	43
5.2	pathlib でパスを扱う	43
5.3	ファイルオブジェクトのメソッド	44
5.4	複数のファイルを集める	44
5.5	1 行の文字列を値へ変換する	45
5.6	ファイルへ書き出す	45
5.7	文字列整形	45
5.8	JSON	46
5.9	Pickle / PKL	46
5.10	日付と時刻を文字列のままにしない	47
5.11	保存形式について: ベストプラクティスへのステップ	47
5.11.1	Step 1: テキスト形式 (CSV, JSON) の限界	47
5.11.2	Step 2: バイナリ形式への移行と、どれを選ぶかの判断基準	48
第 6 章	NumPy による数値計算	49
6.1	なぜ NumPy を使うのか	49
6.2	NumPy が速くなりやすい理由	49
6.3	よく使う操作	50
6.4	形とブロードキャスト	50
6.5	統計量とヒストグラム	51
6.6	配列処理の注意	51
6.7	なぜ NumPy を使うのか: リストの限界	52
6.7.1	Step 1: Python リストの素朴なアプローチ	52
6.7.2	Step 2: NumPy のベクタライズ化	52
6.8	Pandas DataFrame との変換	52
第 7 章	pandas による大規模データ解析	55
7.1	DataFrame とは何か	55
7.2	列選択、並べ替え、集約	55
7.3	pandas が速くなりやすい理由	56
7.4	大きなテキストを読む工夫	56
7.5	CSV を毎回読み直さず Parquet へ変換する	57
7.6	欠損値と型の扱い	57

7.7	結合と時系列処理	58
7.8	DataFrame を TeX で出力する	58
7.9	Pandas の行処理と並列化：論理的改善ステップ	59
7.9.1	Step 1: 絶対に避けるべき for ループ (iterrows の限界)	59
7.9.2	Step 2: Pandas ネイティブな apply() への移行	59
7.9.3	Step 3: 並列化とボトルネックの判断 (Pandallel)	60
7.10	さらに大規模なデータへの対応 (Polars など)	61
第 8 章	Matplotlib による可視化	63
8.1	図は結果そのものである	63
8.2	最小限のヒストグラム	63
8.3	オブジェクト指向 API で描く	63
8.4	見た目を整える	64
8.5	rcParams で見た目をそろえる	64
8.6	設定ファイル (.mplstyle) で見た目を共通化する	65
8.7	共通スタイルの一部だけを上書きする	65
8.8	Matplotlib の数式表記を使う	66
8.9	凡例、余白、複数パネル	66
8.10	結果例の読み方	67
8.11	画像フォーマットの選択：ラスタとベクタ	68
8.12	関連の可視化：論理的改善ステップ	68
8.12.1	Step 1: 直感的な散布図 (Scatter) の限界	68
8.12.2	Step 2: 2 次元ヒストグラムの導入と、線形スケールの限界	69
8.12.3	Step 3: 大規模データに必須となる LogNorm (対数カラスケール)	69
8.13	複数の図を並べる (subplots)	71
8.14	エラーバーと対数軸 (log-log プロット)	71
第 9 章	SciPy とフィット	73
9.1	フィットとは何か	73
9.2	この本でよく使う分布	73
9.3	SciPy の使い方	73
9.4	Astropy で時刻を扱う	74
9.5	Astropy で単位を扱う	75
9.6	フィットの実務的な流れ	75
9.7	初期値と制約	76
9.8	残差とフィット範囲	76
9.9	フィット時の sigma(重み) の重要性	77
9.9.1	Step 1: 誤差を無視した素朴なフィットの限界	77
9.9.2	Step 2: sigma による適切な重み付け	77

第 10 章	Python 解析の高速化	79
10.1	まず測る	79
10.2	Jupyter では <code>%timeit</code> も使える	79
10.3	計算量の見積もり	80
10.4	遅い例と速い例を比べる	80
10.5	ペア探索の遅い例	80
10.6	ペア探索の速い例	81
10.7	トリプレット探索を題材にしたアルゴリズムの差	81
10.8	NumPy と pandas による高速化	81
10.9	メモリとコピーの管理	81
10.10	分割読込み、並列化、ストリーミング	82
10.11	プロファイラを使う	82
10.12	C や C++ へ書き換えるという選択肢	82
10.13	Python から C/C++ を呼び出す	83
10.14	外部実装を参考にするときの見方	83
10.15	結果例と報告の仕方	84
第 11 章	課題: 同時観測イベントの探索	87
11.1	配布ファイル	87
11.2	まず配布コードを動かす	87
11.3	gzip ファイルの中身を見る	87
11.4	小さいデータで確認する	88
11.5	課題でやってほしいこと	88
11.6	出力としてほしいもの	89
11.7	正しさ確認の目安	89
11.8	提出物	90
11.9	発展課題	91
付録 A	SSH, 鍵認証, ファイル転送	93
A.1	SSH とは何か	93
A.2	サーバーへ接続する	93
A.3	SSH 鍵を作る	93
A.4	公開鍵をサーバーへ登録する	94
A.4.1	<code>ssh-copy-id</code> が使えない場合	94
A.5	<code>ssh-agent</code> を使う	94
A.6	<code>~/.ssh/config</code> を書く	95
A.7	GitHub に SSH 鍵を登録する	95
A.8	ファイル転送は <code>rsync</code> を基本にする	95
A.8.1	<code>scp</code> の最小例	95

A.8.2	rsync を推奨する理由	96
A.8.3	末尾のスラッシュに注意する	96
付録 B	Git と GitHub の基本	97
B.1	Git と GitHub は別物である	97
B.2	リポジトリと公開範囲	97
B.3	Git で何をしているのか	97
B.4	最初の設定	98
B.5	新しいリポジトリを作る	98
B.6	編集、確認、記録の流れ	98
B.6.1	git status の読み方	98
B.6.2	git add は何をしているのか	99
B.7	.gitignore を使って除外する	99
B.7.1	.gitignore を書く	99
B.7.2	すでに追跡済みなら git rm --cached が必要	99
B.7.3	uv プロジェクトで何をコミットするか	100
B.8	既存のリポジトリを持ってくる	100
B.9	手元のリポジトリを GitHub へ初めて載せる	100
B.9.1	GitHub 側で空のリポジトリを作る	100
B.9.2	README や LICENSE を先に作っていた場合	100
B.9.3	接続先を登録して最初の push を行う	101
B.9.4	origin がすでにある場合	101
B.10	GitHub と同期する	102
B.11	GitHub で使う URL には HTTPS と SSH がある	102
B.12	GitHub の認証: PAT と SSH 鍵	102
B.12.1	PAT を使う	102
B.12.2	SSH 鍵を使う	103
B.13	初心者が押さえるべき最小ワークフロー	103

はじめに

このテキストは、Python を使ってデータを読み、整形し、図を描き、分布を調べるための入門書である。単に文法を覚えるのではなく、結果の正しさを自分で確かめ、処理時間を測り、改善点を説明できるようになることを目標にする。

本書の構成は Python 公式チュートリアルの流れを意識している。前半では、変数、リスト、関数、ファイル読み書きといった基礎を確認する。中盤では NumPy、pandas、Matplotlib、SciPy、Astropy を用いた実践的な解析へ進む。後半では、大規模データの高速化や低レベル言語との連携に触れ、最後に総合演習へ進む。

特に、Python 公式チュートリアルの「An Informal Introduction to Python」「More Control Flow Tools」「Data Structures」「Modules」「Input and Output」「Errors and Exceptions」「Brief Tour of the Standard Library」で扱われている基礎事項を、データ解析の文脈でつなぎ直すことを意識している。公式チュートリアルを置き換えるというより、その内容を土台にして、解析実装で本当に困る点まで掘り下げることを目指す。

コマンド例は、macOS 上のターミナルで作業する状況を自然な基準として書いている。Terminal.app でも iTerm2 でも構わない。Windows ユーザーは PowerShell を用いてもよいが、Windows Subsystem for Linux (WSL) の使用を強く推奨する。

第 1 章 このテキストの使い方

1.1 誰のためのテキストか

このテキストは、次のような読者を想定している。

- Python をこれから学びたい人
- 多少は書けるが、解析コードをどう整理すればよいか分からない人
- 小さなデータでは動くが、大きなデータになると急に遅くなる理由を知りたい人
- 結果の図をそれらしく描くことはできても、正しさの確認やフィットの説明に自信がない人

逆に、すでに Python で大規模解析を日常的に行っており、NumPy や pandas の内部挙動にも慣れている読者にとっては、前半の内容は基礎的に見えるだろう。その場合は、後半の高速化や総合演習から読んでも構わない。

本書は、単に「使い方を覚える」ためだけの資料ではない。研究や課題で自力の解析を組み立てるとき、何を順に確かめればよいか、どの段階で道具を変えるべきかまで含めて学ぶことを目指している。したがって、短いサンプルコードであっても、なぜその書き方になっているのかを説明する。

1.2 何を順に学ぶのか

本書では、大きく分けて三段階で学ぶ。

第一段階では、Python の基本的な書き方と、ファイルを読むための最低限の技術を扱う。ここでは、短いスクリプトを自分で動かし、何が入力で何が出力なのかをはっきり意識する。

第二段階では、NumPy、pandas、Matplotlib、SciPy を使い、実際の解析に必要な表現力を身につける。配列、表形式データ、ヒストグラム、フィットといった道具が、この段階の中心になる。

第三段階では、高速化と設計を学ぶ。遅い実装と速い実装を比べ、どこで時間を使っているのかを測り、アルゴリズムの違いが速度にどう響くのかを考える。この延長として、Python だけでなく C や C++ へ処理を移す発想も扱う。

この順序には理由がある。NumPy や pandas を早く使いたくなるのは自然だが、入力と出力、型、反復、条件分岐が曖昧なままでは、便利なライブラリを使っても何が起きているか分からなくなる。基礎を先に固めるのは遠回りではなく、後半を理解する最短経路である。

1.3 本書で大切にしたい姿勢

本書で繰り返し強調するのは、次の三点である。

- 結果をそのまま信じず、自分で確かめること
- 遅いと感じたら、どこが遅いかを測ること
- コードが動く理由と、速くなる理由を言葉で説明すること

特にデータ解析では、コードが実行できたことと、結果が正しいことは同じではない。さらに、正しいコードであっても、データ量が増えた途端に実用にならなくなることがある。本書では、その両方を区別して考える。

もう一つ大切なのは、結果を言葉に直すことである。たとえば「NumPy を使ったら速くなった」と書くだけでは不十分で、どの操作を Python ループから配列演算へ移したのか、どの程度速くなったのか、メモリ使用量はどう変わったのか、といった説明が必要になる。レポートや口頭説明では、この差がそのまま理解の差として現れる。

1.4 章の読み進め方

各章には、できるだけ次の順序で内容を置く。

1. その章で扱う考え方の意味
2. 最小限のコード例
3. よくある落とし穴
4. 解析へのつながり

最初は、コードを丸ごと理解しようとしなくてよい。まずは「何を入力し、何を出しているか」を追う。その後で「なぜこの書き方を選んだか」を読むと、理解しやすい。

特に初学者は、1 行ずつ読んで意味を取ろうとして全体像を見失いがちである。まずは関数名、引数、返り値、ファイル名、出力先だけを追い、流れをつかんでから細部へ戻る方が理解しやすい。本書のコード例も、その読み方を意識して配置している。

1.5 併用するとよい公開資料

本書は独立して読めるように書いているが、コマンドの細かいオプションやライブラリの仕様は版によって変わることがある。そのため、必要に応じて公式の公開資料も併用した方がよい。本書の記述も、以下のような公開資料の内容を踏まえて整理している。

- Python 全般: [Python 公式チュートリアル](#) と [Python 標準ライブラリ](#)
- JupyterLab: [JupyterLab Documentation](#)

- uv: [uv Documentation](#)
- pyenv: [pyenv README / Documentation](#)
- NumPy: [NumPy Documentation](#)
- pandas: [pandas Documentation](#)
- Matplotlib: [Matplotlib Documentation](#)
- SciPy: [SciPy Documentation](#)
- Astropy: [Astropy Documentation](#)
- Git: [Git Documentation](#)
- GitHub: [GitHub Docs](#)

特にシェルの `ssh`、`rsync`、`ls`、`chmod` などは、環境ごとに細かな違いがある。細部を確認したいときは `man ssh` や `man rsync` のようにマニュアルを開く習慣も有効である。

1.6 この本の後半で扱う実践的な内容

本書の後半では、単なる文法紹介ではなく、実際のデータを扱うときに何が問題になるかを扱う。そこでは、正しさの確認、バグの切り分け、図の作成、フィット、速度改善といった要素が一つの流れとして現れる。

最後の章には具体的な総合演習を置くが、それは本書で扱った一般的な考え方をまとめて使うための実践例という位置付けである。したがって、前半の章では特定の演習に閉じず、より広いデータ解析へそのまま応用できる書き方と考え方を重視する。

言い換えると、後半へ行くほど「何を書くか」よりも「どう考えるか」が主題になる。ライブラリの関数名は将来変わることがあるが、入力を確認する、分布を見る、速度を測る、遅い理由を切り分ける、といった考え方は長く残る。

第2章 シェルとファイル操作

2.1 ターミナルとシェル

本書では、ファイル操作やスクリプト実行をターミナルから行う。まず言葉を分けておく。

- ターミナル: 文字で命令を入力するためのアプリケーション
- シェル: ターミナルの中で動き、入力されたコマンドを解釈して実行するプログラム

macOS では `zsh`、Linux や WSL では `bash` を使うことが多い。`zsh` と `bash` はどちらもシェルであり、本書の基本コマンドはほぼ同じように使える。

リスト 2.1 どのシェルを使っているか調べる

```
$ echo $SHELL
```

本書では、シェルで入力するコマンド例の先頭に `$` を付ける。`$` 自体は「ここから先が入力するコマンドである」という印であり、読者が実際に打ち込むのはその後ろの部分である。これに対して、出力結果やファイル内容の例には `$` を付けない。

2.2 Windows では WSL を使う

本書のコマンド例は macOS や Linux のシェルを前提にしている。そのため、Windows で読み進める場合は、コマンドプロンプトや PowerShell をそのまま使うより、WSL (Windows Subsystem for Linux) を入れて Ubuntu などの Linux 環境を使う方が混乱しにくい。

リスト 2.2 WSL の導入例

```
$ wsl --install
```

WSL を使う利点は、単にコマンド名がそろうことだけではない。後半で扱う並列処理や周辺ツールの中には、Linux 系環境での利用を前提としているものがある。OS ごとの差でつまづくより、最初から UNIX 系の作法へ寄せた方が、解析そのものに集中しやすい。

2.3 コマンド、オプション、引数

シェルで打つ 1 行は、多くの場合「コマンド本体」「オプション」「引数」から成る。

リスト 2.3 コマンドの形

```
$ ls -l results
$ cp data/raw.txt backup/raw.txt
```

1 行目では、`ls` がコマンド本体、`-l` がオプション、`results` が引数である。2 行目では、`cp` がコマンド本体で、`data/raw.txt` と `backup/raw.txt` が引数である。一般に、オプションは動作を変え、引数は対象のファイル名やディレクトリ名を与える。

2.4 困ったときは説明を読む

コマンドを丸暗記する必要はない。忘れたときは、その場で説明を読む方が正確である。短い使い方を知りたいときは `-h` や `--help`、より詳しい説明を読みたいときは `man` を使う。

リスト 2.4 説明を調べる基本

```
$ python3 -h
$ python3 --help
$ man rsync
```

`--help` や `-h` は、そのコマンドが受け付けるオプションや引数の書式を短く表示することが多い。ただし、すべてのコマンドが両方を実装しているわけではない。macOS の `ls` のように `--help` を受け付けないコマンドもあるので、その場合は `man ls`、`man ssh`、`man rsync` のように `man` を引く方が確実である。

`man` を開いたあとは、上下キーや `Space` で移動し、`/pattern` で検索できる。q を押せば終了する。最初のうちは「使い方を忘れたら検索エンジン」ではなく、「まず `--help` か `man`」という順序にした方が、手元の環境に合った説明へすぐたどり着ける。

2.5 今どこにいるかを確認する

シェルでは、いま作業しているディレクトリが常に基準になる。これをカレントディレクトリという。

リスト 2.5 現在位置と一覧を確認する

```
$ pwd
$ ls
$ ls -l
$ ls -a
```

`pwd` は現在位置を表示する。`ls` は一覧を表示する。`ls -l` は詳細表示、`ls -a` は隠しファイルも含めて表示する。

リスト 2.6 表示例

```
/Users/ikomae/analysis
```

もし `pwd` が上のように表示されていれば、今いる場所は `/Users/ikomae/analysis` である。その状態で `ls` を実行すると、そのディレクトリの中身が表示される。

2.6 移動する

`cd` はディレクトリを移動するためのコマンドである。次のようなディレクトリ構成を例にする。

リスト 2.7 例に使うディレクトリ構成

```
analysis/  
|-- data/  
| `-- events.txt  
`-- scripts/  
    |-- plot.py
```

いま `/Users/ikomae/analysis` にいるとする。このとき、次のコマンドの意味はそれぞれ違う。

リスト 2.8 `cd` の例

```
$ cd data  
$ cd ..  
$ cd ~/analysis/scripts  
$ cd -
```

`cd data` は `analysis/data` へ入る。`cd ..` は 1 つ上のディレクトリへ戻る。`cd ~/analysis/scripts` はホームディレクトリを基準に `scripts` へ移動する。`cd -` は 1 つ前にいた場所へ戻る。

2.7 パスを読む

ファイルやディレクトリの場所は、パスで表す。パスには絶対パスと相対パスがある。

表 2.1 よく出てくるパス表記

表記	意味
<code>/abs/data.txt</code>	絶対パス。どこにいても同じ場所を指す
<code>data.txt</code>	相対パス。今いるディレクトリを基準に解釈される
<code>./x.txt</code>	<code>.</code> は現在のディレクトリ
<code>../x.txt</code>	<code>..</code> は 1 つ上のディレクトリ
<code>~/analysis</code>	<code>~</code> はホームディレクトリ

たとえば、現在位置が `/abs/analysis` なら、次の 2 つは同じファイルを指す。

リスト 2.9 同じファイルを指す 2 通りの書き方

```
$ cat data/events.txt  
$ cat /abs/analysis/data/events.txt
```

一方で、現在位置が `/abs` なら、`cat data/events.txt` は別の意味になる。したがって、コマンドを読むときは「今どこにいるか」と「そのパスが絶対か相対か」を必ずセットで考える必要がある。

2.8 ディレクトリを作る

`mkdir` はディレクトリを作る。`-p` を付けると、親ディレクトリがなくてもまとめて作れる。

リスト 2.10 `mkdir` の例

```
$ mkdir output
$ mkdir -p output/2026/04
```

1 行目は `output` だけを作る。2 行目は `output` や `output/2026` がまだ無くても、必要な親ディレクトリごと作る。解析では日付ごとの出力先を作ることが多いので、`mkdir -p` はよく使う。

2.9 ファイルの中身を見る

解析では、データを読む前にまず中身を確認する。そのための基本コマンドを表にまとめる。

表 2.2 中身確認でよく使うコマンド

コマンド	役割	例
<code>cat</code>	全体をそのまま表示する	<code>cat data/events.txt</code>
<code>less</code>	スクロールしながら読む。終了は <code>q</code>	<code>less data/events.txt</code>
<code>head</code>	先頭だけを見る	<code>head data/events.txt</code>
<code>tail</code>	末尾だけを見る	<code>tail data/events.txt</code>
<code>wc -l</code>	行数を数える	<code>wc -l data/events.txt</code>

巨大なファイルに対して `cat` を使うと画面が流れてしまうので、その場合は `head`、`tail`、`less` を使う方が自然である。`less` を開いたあとに元のシェルへ戻りたいときは `q` を押す。データ解析では、まず数行だけ見て形式を確認する癖を付けた方がよい。

2.10 コピーする: `cp`

`cp` は元のファイルやディレクトリを残したまま複製を作る。文章だけで覚えると混乱しやすいので、前後の状態を見た方がよい。

リスト 2.11 コピー前の状態

```
work/  
|-- backup/  
`-- raw/  
    |-- events.txt
```

リスト 2.12 ファイルをコピーする

```
$ cp raw/events.txt backup/events.txt
```

リスト 2.13 コピー後の状態

```
work/  
|-- backup/  
| `-- events.txt  
`-- raw/  
    |-- events.txt
```

コピー後も raw/events.txt は残る。backup/events.txt が新しく増えるだけである。これが cp の基本である。

ディレクトリごと複製したい場合は -r を付ける。

リスト 2.14 ディレクトリごとコピーする

```
$ cp -r raw raw_backup
```

2.11 移動する、名前を変える: mv

mv は「移動」と「改名」の両方に使う。どちらになるかは、最後の引数が何を指しているかで決まる。

2.11.1 改名の例

リスト 2.15 改名前の状態

```
work/  
`-- result.txt
```

リスト 2.16 ファイル名を変更する

```
$ mv result.txt result_old.txt
```

リスト 2.17 改名後の状態

```
work/  
`-- result_old.txt
```

この場合は、同じディレクトリの中で名前だけが変わる。

2.11.2 移動の例

リスト 2.18 移動前の状態

```
work/  
|-- output/  
`-- result.txt
```

リスト 2.19 別ディレクトリへ移動する

```
$ mv result.txt output/
```

リスト 2.20 移動後の状態

```
work/  
`-- output/  
    |-- result.txt
```

この場合は、`result.txt` が `output/` の中へ移る。元の場所からは消える。`cp` と違って元ファイルは残らない。

2.12 削除する: `rm`

`rm` はファイルを削除する。重要なのは、通常の `rm` はゴミ箱へ移すのではなく、その場で削除するという点である。

リスト 2.21 削除の例

```
$ rm result_old.txt  
$ rm -i result_old.txt  
$ rm -r old_output
```

`rm result_old.txt` はファイルを削除する。`rm -i result_old.txt` は削除前に確認を出す。`rm -r old_output` はディレクトリを中身ごと再帰的に削除する。

特に `rm -r` は強力である。対象のパスを誤ると大きな被害になるので、削除前に `pwd` と `ls` で場所を確かめる習慣を持った方がよい。

2.13 環境変数を使う

シェルやプログラムには、「名前」と「値」の組として渡される設定がある。これが環境変数である。環境変数は、シェルの中だけで完結するものではなく、そこから起動した Python や各種コマンドにも引き継がれる。

リスト 2.22 よく使う環境変数を見る

```
$ echo $HOME  
$ echo $SHELL  
$ echo $PATH
```

HOME はホームディレクトリ、SHELL は使っているシェル、PATH は実行ファイルを探すディレクトリの並びを表す。後で出てくる PYENV_ROOT や DATA_ROOT も同じ仲間であり、「プログラムへ外から値を渡す仕組み」と考えると分かりやすい。

リスト 2.23 環境変数を設定する

```
$ export DATA_ROOT="$HOME/data/events"
$ echo $DATA_ROOT
```

export すると、そのシェルから起動する後続のコマンドにも値が渡る。たとえば Python スクリプト側で DATA_ROOT を読めば、データ置き場をコードへ埋め込まずに済む。

ただし、この設定は通常そのターミナルを閉じると消える。毎回使う値なら、~/.zshrc や ~/.bashrc に export ... を書いておく。

2.14 PATH と実行ファイル

シェルが python3 や ls を実行できるのは、それらが実行ファイルとして保存されており、しかもシェルがその場所を知っているからである。その探索に使う環境変数が \$PATH である。

リスト 2.24 実行ファイルを探す

```
$ which python3
$ which ls
$ echo $PATH
```

which python3 を実行すると、たとえば /opt/homebrew/bin/python3 や .venv/bin/python のような場所が表示される。シェルは \$PATH に並んだディレクトリを左から順に調べ、最初に見つかった実行ファイルを使う。

bin という名前は、実行ファイルを置くディレクトリによく使われる。後で出てくる .venv/bin/python や .venv/bin/pip の bin も、この慣習に従っている。

2.15 権限を読む

ファイルには権限がある。読み取れるか、書き込めるか、実行できるかを rwx で表す。ls -l を使うと確認しやすい。

リスト 2.25 権限表示の例

```
-rwxr-xr-x 1 user staff 1234 Apr 8 10:00 run_analysis.sh
-rw-r--r-- 1 user staff 411 Apr 8 10:05 notes.txt
-rw----- 1 user staff 411 Apr 8 10:05 id_ed25519
```

最初の 10 文字のうち、先頭 1 文字はファイル種別であり、残り 9 文字が権限である。3 文字ずつに区切って、所有者、グループ、その他の利用者に対する権限を表す。

表 2.3 権限の記号と数字

記号	数字	意味
r	4	読み取り
w	2	書き込み
x	1	実行

数字指定では足し算で表す。たとえば 7 は rwx、6 は rw-、5 は r-x である。したがって 755 は所有者が rwx、それ以外が r-x を意味し、644 は所有者が rw-、それ以外が r-- を意味する。秘密鍵のように本人だけが読めればよいものには 600 がよく使われる。

リスト 2.26 権限を変更する例

```
$ chmod 755 run_analysis.sh
$ chmod 644 notes.txt
$ chmod 600 ~/.ssh/id_ed25519
```

2.16 スクリプトを直接実行する

Python スクリプトは `python3 hello.py` の形で実行するなら、実行権限がなくてもよい。これに対して `./hello.py` のように直接実行する場合には、どのプログラムで解釈するかをファイル先頭に書いておく必要がある。

その先頭行の `#!` で始まる記法を、シバン (shebang) という。

リスト 2.27 シバン付きの Python スクリプト

```
1 #!/usr/bin/env python3
2
3 print("hello")
```

リスト 2.28 直接実行できるようにする

```
$ chmod +x hello.py
$ ./hello.py
```

`#!/usr/bin/env python3` は、「python3 を見つけてこのファイルを実行せよ」という意味である。いきなり言葉だけで覚える必要はないが、「直接実行にはシバンと実行権限の両方が必要だ」という構造だけは理解しておいた方がよい。

第 3 章 Python 環境を整える

3.1 何を分けて考えるべきか

Python を使い始めるときは、次の 3 つを混同しない方がよい。

1. Python 本体を入れること
2. 複数版の Python を切り替えること
3. プロジェクトごとの仮想環境を作ること

まず Python 本体が必要であり、そのうえで Python 3.11 と 3.12 を切り替えたいなら版管理ツールが必要になる。さらに、同じ Python 3.12 でもプロジェクト A と B で依存ライブラリが違えば、仮想環境で分ける必要がある。この 3 段階を分けて理解した方が、あとで混乱しにくい。

3.2 今の Python を確認する

macOS でも WSL でも、最初にいま見えている Python を調べておく方がよい。

リスト 3.1 Python の確認

```
$ python3 --version
$ which python3
$ python3 -c "import sys; print(sys.executable)"
```

`python3 --version` で版を確認し、`which python3` と `sys.executable` で実体の場所を確認する。複数版を切り替えるようになると、この確認は非常に重要になる。

3.3 pyenv で複数版の Python を管理する

Python 本体の導入と切り替えには `pyenv` が分かりやすい。`pyenv` は、シェルから見える `python` をディレクトリ単位で切り替えるための道具であり、複数版の Python を使い分けるときの基準として扱いやすい。

3.3.1 macOS での準備

`pyenv` は Python 本体をビルドして入れるので、macOS では Xcode Command Line Tools が必要になる。

リスト 3.2 Command Line Tools があるか確認する

```
$ xcode-select -p
```

すでに Homebrew が正常に動いているなら、Command Line Tools は入っていることが多い。上のコマンドでパスが表示されれば、そのまま進んでよい。表示されない場合は次を実行する。

リスト 3.3 Command Line Tools を入れる

```
$ xcode-select --install
```

3.3.2 WSL / Ubuntu での準備

WSL の Ubuntu では、ビルドに必要なライブラリを先に入れておく。

リスト 3.4 Ubuntu での準備

```
$ sudo apt update
$ sudo apt install -y build-essential curl git libssl-dev zlib1g-dev \
  libbz2-dev libreadline-dev libsqlite3-dev libffi-dev liblzma-dev tk-dev
```

3.3.3 pyenv を入れる

リスト 3.5 pyenv の導入

```
$ curl -fsSL https://pyenv.run | bash
```

3.3.4 どの設定ファイルを編集するか決める

pyenv を使うには、シェルの設定ファイルへ初期化コードを書く必要がある。どのファイルを編集するかは、いま使っているシェルで決まる。

リスト 3.6 シェルを調べる

```
$ echo $SHELL
```

/bin/zsh と出たら ~/.zshrc、/bin/bash と出たら ~/.bashrc を編集する。これで、zsh や bash が唐突に出てくる状態は避けられる。

3.3.5 pyenv をシェルへ組み込む

リスト 3.7 zsh での設定

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
$ echo 'eval "$(pyenv init - zsh)"' >> ~/.zshrc
$ exec "$SHELL"
```


リスト 3.8 bash での設定

```
$ echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
$ echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
$ echo 'eval "$(pyenv init - bash)"' >> ~/.bashrc
$ exec "$SHELL"
```

3.3.6 版を入れて切り替える

pyenv を使う場合は、pyenv install で版を追加し、pyenv global、pyenv local、pyenv shell で切り替える。

リスト 3.9 版を追加して確認する

```
$ pyenv install 3.11.11
$ pyenv install 3.12.10
$ pyenv versions
```

リスト 3.10 普段使う版を決める

```
$ pyenv global 3.12.10
$ python --version
$ which python
```

pyenv global は普段使う版を決める。これに対して pyenv local は、そのディレクトリだけ別の版を使いたいときに使う。

リスト 3.11 プロジェクト単位で版を固定する

```
$ mkdir analysis311
$ cd analysis311
$ pyenv local 3.11.11
$ python --version
$ cat .python-version
```

pyenv local を実行すると .python-version が作られ、そのディレクトリでは指定した版が優先される。これによって、同じマシンの中でプロジェクト A は 3.11、プロジェクト B は 3.12 という使い分けができる。

pyenv shell 3.11.11 は、そのターミナルを閉じるまでの一時的な切り替えである。検証用に 1 回だけ古い版で動かしたいときに便利である。

3.4 venv でプロジェクトごとの仮想環境を作る

Python 本体の版を決めたら、その版の上にプロジェクト専用の仮想環境を作る。依存ライブラリの衝突を避けるためである。

リスト 3.12 venv の基本

```
$ python -m venv .venv
```

```
$ source .venv/bin/activate
$ which python
$ which pip
$ python --version
$ pip --version
```

仮想環境へ入ると、プロンプトの先頭に (.venv) が付くことが多い。

リスト 3.13 プロンプトの例

```
(.venv) ikomae@macbook analysis $
```

which python と which pip を見ると、.venv/bin/python と .venv/bin/pip が使われていることが分かる。これは、source .venv/bin/activate が \$PATH の先頭に .venv/bin を追加しているからである。

3.4.1 pip と python -m pip

仮想環境へ入っているなら、通常は pip と python -m pip は同じ環境の pip を指す。したがって、作業中は pip install numpy と書いてよい。

一方で、いまどの Python が有効か不安なときや、スクリプトや文書の中で「この Python に対応する pip を使う」と明示したいときは、python -m pip の方が誤解が少ない。実務では両方見るが、意味の違いを分かって使うことが重要である。

リスト 3.14 仮想環境での導入例

```
$ pip install --upgrade pip
$ pip install numpy pandas matplotlib scipy astropy tqdm
```

作業を終えたら deactivate で仮想環境から抜けられる。

3.5 JupyterLab を併用する

対話的に試しながら解析したいときは、JupyterLab も便利である。小さなコード片をその場で動かし、数値や図をすぐ確認できるため、探索的な作業では特に役立つ。

リスト 3.15 JupyterLab の導入と起動

```
$ pip install jupyterlab
$ jupyter lab
```

JupyterLab の利点は、コードを実行できるだけでなく、その前後に説明文、数式、図の読み方を書き添えられる点にある。解析の途中経過を他人へ見せるときも、自分で後から読み返すときも、コードと説明が同じノートブックに並んでいる方が理解しやすい。

リスト 3.16 Markdown セルの例

```
# 解析メモ

- 入力ファイル: `events_2024_01_01.txt.gz`
```

```
- 閾値: `signal > 5.0`
```

```
$\Delta t$ の分布を確認する
```

ただし、JupyterLab は最終成果物の置き場所ではなく、試行錯誤の場として使う方が整理しやすい。解析手順を再現したい段階では、ノートブック中で固めた処理を Python スクリプトへ戻し、入力や出力を明示した方が後で困りにくい。

3.6 uv はプロジェクト管理に使う

uv は Python 本体の導入もできるが、本書ではそれよりも「プロジェクト単位の管理」に使う道具として位置付ける方が自然である。特に、依存関係、ロックファイル、`.python-version`、Git 管理、パッケージ化、ワークスペース管理といった、プロジェクト全体の整理で力を発揮する。

3.6.1 まずは既定の uv init を使う

多くの人は、まずシンプルに `uv init` を使えばよい。オプションは後から必要になったときに足せばよい。

リスト 3.17 uv で新しいプロジェクトを始める

```
$ mkdir analysis_uv
$ cd analysis_uv
$ uv init
```

既定の `uv init` では、少なくとも次のようなファイルやディレクトリが作られる。

リスト 3.18 既定の uv init が作るもの

```
.git/
.gitignore
.python-version
README.md
main.py
pyproject.toml
```

つまり、`uv init` は単に Python ファイルを 1 つ置くだけでなく、Git 管理の土台までまとめて用意する。この点が `venv` 単体との大きな違いである。

3.6.2 必要ならオプションを使う

既定動作を変えたいときだけオプションを使えばよい。

- `uv init --bare`: `pyproject.toml` だけを作る
- `uv init --vcs none`: Git 初期化をしない
- `uv init --no-pin-python`: `.python-version` を作らない

- `uv init --package` や `uv init --lib`: スクリプトではなくパッケージやライブラリとして始める

特に、コードを再利用可能なモジュールやライブラリとして育てたいなら、`--package` や `--lib` を意識した方がよい。単発スクリプトで終わるのか、将来 `import` される部品になるのかで、適切な初期構成は変わる。

3.6.3 依存関係を追加する

`uv add` は依存ライブラリを追加し、`pyproject.toml` とロック情報を更新する。

リスト 3.19 依存関係を追加する

```
$ uv add numpy pandas matplotlib scipy astropy tqdm
```

3.6.4 環境をそろえる

`uv sync` は、`pyproject.toml` とロックファイルに基づいて環境をそろえる。ここで初めて `.venv` が作られることもある。

リスト 3.20 環境を同期する

```
$ uv sync
```

過去の自分の解析や、他人から受け取ったプロジェクトを再現するときは、この `uv sync` が重要になる。リポジトリを取得して `uv sync` すれば、必要な依存関係をまとめてそろえやすい。ここで Git の基本が必要になるので、付録 B もあわせて読むとよい。

3.6.5 コマンドを実行する

`uv run` は、そのプロジェクトの環境でコマンドを実行する。スクリプトを直接走らせるときと、Python 本体を起動したいときとで書き分ける。

リスト 3.21 `uv run` の例

```
$ uv run main.py
$ uv run python --version
$ uv run python -m pytest
```

実行対象が Python スクリプトそのものであるなら、`uv run main.py` と書けばよい。わざわざ `python` を挟む必要はない。一方で、`-m` を使いたいときや、Python 本体へオプションを渡したいときは `uv run python ...` と書く。

3.6.6 Python 版の固定にも使える

`uv` もプロジェクト単位で Python 版を固定できる。`pyenv` がシェル全体の切り替えを担当し、`uv` がプロジェクト側の固定を担当する、と分けて考えると整理しやすい。

リスト 3.22 `uv` で Python 版を固定する

```
$ uv python pin 3.12
$ uv run python --version
$ uv run --python 3.11 python --version
```

3.7 標準ライブラリと外部ライブラリ

Python には、最初から使える標準ライブラリと、自分で追加する外部ライブラリがある。

- 標準ライブラリの例: `math`, `pathlib`, `glob`, `argparse`
- 外部ライブラリの例: `numpy`, `pandas`, `matplotlib`, `scipy`, `astropy`

標準ライブラリは Python 本体に含まれている。外部ライブラリは、目的に応じて自分で追加する。解析では外部ライブラリが非常に重要だが、まずは「何が標準で、何が追加なのか」を意識しておくとうれしい。

3.8 解析用のライブラリを入れる

本書の後半で使う主なライブラリを表 3.1 に示す。

表 3.1 本書で使う主なライブラリと役割	
ライブラリ	主な役割
NumPy	数値配列の高速処理
pandas	表形式データの読み込みと加工
Matplotlib	図の作成
SciPy	フィットと数値計算
Astropy	時刻・単位・座標の扱い
tqdm	進捗表示

`venv` を使うなら `pip install`、`uv` を使うなら `uv add` と書けばよい。

リスト 3.23 `venv` を使う場合

```
$ pip install numpy pandas matplotlib scipy astropy tqdm
```

リスト 3.24 `uv` を使う場合

```
$ uv add numpy pandas matplotlib scipy astropy tqdm
```

3.9 コマンドライン引数と設定ファイルを使い分ける

同じ解析スクリプトを何度も使うなら、入力ディレクトリや出力先をコードに埋め込むより、コマンドライン引数で渡す方がよい。Python では `argparse` が標準で使える。

リスト 3.25 argparse の最小例

```
1 import argparse
2
3 parser = argparse.ArgumentParser()
4 parser.add_argument("input_dir")
5 parser.add_argument("--outdir", default="output")
6 args = parser.parse_args()
7
8 print(args.input_dir, args.outdir)
```

ただし、何でも引数にすればよいわけではない。毎回ほぼ同じ値まで毎回書かせると、使いにくいスクリプトになる。

値の置き場所は、表で機械的に覚えるより、用途ごとに言葉で整理した方が分かりやすい。

コマンドライン引数 実行ごとに変わる値を置く。入力ディレクトリ、出力先、日付範囲、並列数などが典型である。

設定ファイル プロジェクト単位では固定だが、将来変更し得る値を置く。校正ファイルのパス、検出器 ID、閾値、図の保存形式などが当てはまる。

環境変数 マシン依存の値や秘密情報を置く。DATA_ROOT、API トークン、認証情報などが典型である。

コード内の定数 定義として固定したい値や単位変換を書く。NS_PER_SECOND や物理定数のように、意味が仕様として決まっているものを置く。

入力ファイルのパス、出力先、しきい値、作業ディレクトリのように「実行条件」であるものは、原則としてハードコーディングしない方がよい。コードを書き換えずに切り替えられるようにしておく、再利用と再現が楽になる。

3.9.1 設定ファイルを読む

めったに変わらない値は、設定ファイルへまとめる方が使いやすい。形式は JSON でも TOML でもよい。ここでは標準ライブラリだけで扱える JSON の例を示す。

リスト 3.26 設定ファイルの例

```
{
  "calibration_path": "calib/detector_a.csv",
  "coincidence_window_ns": 250,
  "figure_format": "pdf"
}
```

リスト 3.27 JSON 設定を読み込む例

```
1 import json
2 from pathlib import Path
3
4 config = json.loads(Path("config.json").read_text())
5 window_ns = config["coincidence_window_ns"]
```

このようにしておけば、同じコードを保ったまま設定だけを差し替えられる。Python 3.11 以降なら `tomllib` を使って TOML を読むこともできる。どの形式を選ぶにしても、「たまたま変わるがコード本体ではない値」を分離することが大切である。

3.9.2 マジックナンバーを避ける

250 や 3600 のような数値が説明なしにコードへ裸で出てくると、後から読んだ人は意味を判断しにくい。このような数値はマジックナンバーと呼ばれる。詳しくは [4.20 節](#) で述べるが、意味のある数値には名前を付け、実行条件なら引数や設定ファイルへ出す、という整理を早い段階で身に付けた方がよい。

第 4 章 Python の基本

4.1 変数と基本型

Python では、値に名前を付けて使う。たとえば次のように書く。

リスト 4.1 変数と基本型

```
1 n_rows = 12
2 threshold = 3600.0
3 label = "A"
4 use_log = True
```

`n_rows` は整数、`threshold` は浮動小数点数、`label` は文字列、`use_log` は真偽値である。解析では、整数と浮動小数点数を区別して扱う感覚が重要になる。たとえば、件数は整数だが、フィットで得られるパラメータは浮動小数点数であることが多い。

Python は変数宣言を先に書かなくてよいが、その代わり自分で型を意識して読む必要がある。特に 1 と 1.0 の違い、文字列としての '12' と整数としての 12 の違いは、ファイル読み込みや数値計算で頻繁に問題になる。

4.2 数値と演算子

Python は、対話的な計算機として使い始めるのが理解しやすい。四則演算、べき乗、剰余、整数除算など、基本的な演算子はすぐに試せる。

リスト 4.2 数値計算の基本

```
1 2 + 3
2 7 / 3
3 7 // 3
4 7 % 3
5 2 ** 10
```

`/` は浮動小数点数として割り算を行い、`//` は整数部分を返す。観測時間を秒へ直す、ビン番号を求める、インデックスを分ける、といった処理では、この違いが重要になる。特に `//` と `%` は、周期的な区切りやグループ分けでよく現れる。

浮動小数点数の計算では、表示された値が見た目通りに内部で表現されているとは限らない。これは Python 特有の問題ではなく、一般的な計算機の表現上の性質である。後半でフィットや統計量を扱うときにも、この前提を意識しておきたい。

4.3 文字列

ファイル名、日付時刻、ヘッダ、ラベルなど、解析コードでは文字列を扱う場面が多い。文字列は単なる表示用のデータではなく、分割、検索、整形の対象でもある。

リスト 4.3 文字列の基本操作

```
1 name = "detector_A"
2 date = "2026-04-04"
3
4 print(name.upper())
5 print(date.replace("-", "/"))
6 print(name.split("_"))
```

split、replace、strip のような操作は、ファイル読み込みの前処理で頻繁に使う。特に split は、空白区切りやカンマ区切りの 1 行を各列へ分ける最初の道具である。NumPy や pandas を使う前に、この感覚を持っておくとデータ形式の理解が速い。

4.4 リスト、タプル、辞書

複数の値をまとめたいときには、いくつかの基本的な入れ物がある。

リスト 4.4 基本的なデータ構造

```
1 labels = ["A", "B", "C", "D"]
2 point = (12, 18)
3 row = {"id": 42, "value": 3.5, "flag": True}
```

リストは順序付きで変更可能、タプルは順序付きで変更不可、辞書は名前付きの値を持つ。最初のうちは辞書が読みやすいが、大きなデータを大量に処理するときはオーバーヘッドが大きくなることもある。この違いは後の高速化章で再び出てくる。

どの入れ物を選ぶかは、見た目の好みではなく、何をしたいかで決まる。位置でアクセスするならリストやタプル、名前でアクセスするなら辞書が自然である。小さなコードでは差が見えにくいですが、処理量が増えると選択の重みが増してくる。

4.5 リストのメソッド

Python 公式チュートリアルでも詳しく扱われているように、リストにはいくつかの基本メソッドがある。これらを知っていると、簡単な前処理や集約を自然に書ける。

リスト 4.5 リストメソッドの例

```
1 values = [3, 1, 4]
2 values.append(1)
3 values.sort()
4 last = values.pop()
```

`append` は末尾へ追加し、`sort` はその場で並べ替え、`pop` は末尾要素を取り出して削除する。リストメソッドの多くは「その場で変更する」点に注意が必要である。新しいリストを返すのではなく、元のリスト自体を書き換える操作が多いので、どこで状態が変わるかを意識して読む必要がある。

解析コードでは、ファイル一覧の作成、簡単なバッファ、途中結果の蓄積などにリストを使うことが多い。ただし、大量の数値そのものを主役として扱う段階では、後で NumPy 配列へ移った方が自然になることが多い。

4.6 インデックスとスライス

解析では、配列や文字列の一部だけを取り出したい場面が非常に多い。Python では、`[]` の中に位置を書くことで要素を取り出し、`:` を使うことで範囲を切り出せる。

リスト 4.6 インデックスとスライスの例

```
1 values = [10, 20, 30, 40, 50]
2
3 first = values[0]
4 last = values[-1]
5 middle = values[1:4]
```

最初の要素が 0 番目であること、終点の添字は含まれないこと、負の添字で末尾から数えられることは、最初にしっかり覚えておきたい。時刻列や列名の一部を取り出すとき、この基本がそのまま使われる。

文字列に対しても同じ考え方が使える。例えば `hhmmss[0:2]` で時、`hhmmss[2:4]` で分を取り出せる。この一貫性が Python の扱いやすさの一つである。

4.7 条件分岐

条件によって処理を変えるには `if` を使う。

リスト 4.7 条件分岐

```
1 if value <= threshold:
2     print("small")
3 else:
4     print("large")
```

解析コードでは、閾値判定、ファイル名の選別、欠損値の除外など、条件分岐が頻繁に現れる。重要なのは、「なぜその条件で分けているのか」を自分で説明できることだ。

条件分岐が増えて読みにくくなったときは、条件式そのものを変数へ切り出すと分かりやすくなる。たとえば `is_valid_row = value <= threshold` のように一度名前を付けるだけでも、後で読むときの負担がかなり減る。

条件は左から右へ読んで意味が分かる形に書く方がよい。否定が重なる条件や、複数の

and と or が混ざる条件は、括弧や一時変数で整理した方が安全である。解析コードでは、条件の誤読がそのままバグになる。

4.8 反復処理

複数の値を順に調べるには for を使う。

リスト 4.8 for 文の例

```
1 for label in labels:
2     print(label)
```

添字も必要なら enumerate が便利である。

リスト 4.9 enumerate の例

```
1 for i, value in enumerate(labels):
2     print(i, value)
```

一方で、データ点が非常に多いとき、Python レベルの for ループが速度の瓶頸になることがある。本書では後半で、その回避方法を詳しく扱う。

初学者の段階では、まず正しく読める for 文を書くことが先である。速く書くことを意識するのは、その後でよい。読みやすい基本形を知っているからこそ、後で NumPy や pandas へ置き換えたときに、何を省略しているかが分かる。

4.9 range と while

for 文と並んで、反復処理でよく使うのが range と while である。range は連続した整数を順に生成し、while は条件が成り立つ間くり返す。

リスト 4.10 range と while の例

```
1 for i in range(5):
2     print(i)
3
4 count = 0
5 while count < 3:
6     print(count)
7     count += 1
```

range(5) は 0 から 4 までを返す。ファイル番号やビン番号のような添字処理では便利である。一方、while は終了条件を自分で管理する必要があるので、無限ループにしないよう注意が必要である。時刻順に読み進めて、条件を満たす間だけ比較するような処理では while が自然な場合もある。

4.10 break, continue, else

反復処理では、途中で打ち切ったり、ある条件だけ飛ばしたりしたくなる。Python では `break` と `continue` がそのために使える。

リスト 4.11 `break` と `continue` の例

```
1 for value in values:
2     if value < 0:
3         continue
4     if value > threshold:
5         break
6     print(value)
```

`continue` はその回だけを飛ばし、`break` はループ全体を終了する。探索範囲を打ち切る処理では特に重要である。さらに Python には `for ... else` という構文もあり、`break` せずに最後まで回り切ったときだけ `else` 節が実行される。頻出ではないが、見かけても驚かないようにしておくといよい。

4.11 内包表記

短い変換や簡単なフィルタであれば、内包表記を使うとすっきり書ける。

リスト 4.12 リスト内包表記の例

```
1 values = [1, 2, 3, 4, 5]
2 squares = [x * x for x in values]
3 large_values = [x for x in values if x >= 3]
```

内包表記は便利だが、何段階もの条件や複雑な変換を 1 行に押し込むと読みにくくなる。初学者のうちは、読みやすさを失わない範囲で使う方がよい。長くなりそうなら、普通の `for` 文や補助関数へ戻した方が親切である。

4.12 関数を分ける理由

同じ処理を何度も書かないためだけでなく、処理の意味を分けるためにも関数は重要である。

リスト 4.13 関数の例

```
1 def hhmmss_to_seconds(hhmmss):
2     hour = int(hhmmss[0:2])
3     minute = int(hhmmss[2:4])
4     second = int(hhmmss[4:6])
5     return hour * 3600 + minute * 60 + second
```

この関数は短いが、「文字列で与えられた時刻を秒へ変換する」という役割が明確である。解析コードでは、このような小さな関数が積み重なって全体の読みやすさを作る。

関数を切り出す利点は再利用だけではない。入力は何で、出力が何かをはっきりさせることで、テストしやすくなる。短い補助関数ほど後回しにされがちだが、解析の正しさはこうした小さな部品の積み重ねで決まることが多い。

4.13 関数の引数

関数を書くときは、引数の意味をはっきりさせることが重要である。Python では位置引数だけでなく、デフォルト値付き引数やキーワード引数も使える。

リスト 4.14 デフォルト引数の例

```
1 def histogram_path(name, suffix=".pdf"):
2     return f"{name}-{suffix}"
3
4 print(histogram_path("value_hist"))
5 print(histogram_path("value_hist", suffix=".png"))
```

デフォルト引数を使うと、よく使う設定を省略できる一方で、必要なら上書きできる。コマンドライン引数と同じで、解析コードの柔軟性は「固定値をどこに置くか」で決まることが多い。関数でも同じ発想が使える。

ただし、可変オブジェクトをデフォルト引数に置くと予想外の共有が起きる。この罠は Python 公式チュートリアルでも強調されている典型例である。初学者の段階では、リストや辞書をデフォルトにしたいくなったら一度立ち止まった方がよい。

リスト 4.15 避けた方がよいデフォルト引数

```
1 def bad_append(value, buffer=[]):
2     buffer.append(value)
3     return buffer
```

この例では、`buffer` が呼び出しのたびに新しく作られるわけではない。そのため、前回の状態が次回にも残る。安全に書くなら `None` を使って関数内部で初期化する方がよい。初学者がつまずきやすいので、早めに知っておく価値が高い。

4.14 モジュールと import

別ファイルに書かれた関数を使うには `import` する。

リスト 4.16 import の例

```
1 import math
2 import glob
3 import argparse
```

Python の解析では、標準ライブラリと外部ライブラリを自然に組み合わせる。どの名前がどこから来たものかを意識して読む習慣を付けておくと、大きなコードも追いやすくなる。

特に、同じ名前に見えても標準ライブラリと外部ライブラリで役割が大きく異なることがある。インポート文は単なるおまじないではなく、そのファイルが何に依存しているかを示す地図でもある。

4.15 自作モジュールへ分ける

解析が少し大きくなると、1 つのファイルへ全部を書くのはすぐに苦しくなる。よく使う関数や定数は別ファイルへ分け、モジュールとして読み込む方が見通しが良い。

リスト 4.17 モジュール分割のイメージ

```
analysis_project/  
|-- constants.py  
|-- physics.py  
`-- main.py
```

リスト 4.18 別ファイルへ分けた関数を使う例

```
1 # physics.py  
2 import numpy as np  
3  
4 def deg_to_rad(degrees):  
5     return degrees * np.pi / 180.0  
6  
7  
8 # main.py  
9 import physics  
10  
11 theta = physics.deg_to_rad(45.0)
```

この形にしておくと、座標変換、時刻変換、定数定義のような再利用しやすい処理を 1 か所へまとめられる。後半で大きめの課題に進むと、コードを速くすることと同じくらい、どこに何を書くかを整理することが重要になる。

4.16 スクリプトとして実行する

Python ファイルは、他のファイルから読み込まれることもあれば、直接実行されることもある。その違いを区別する基本形が次である。

リスト 4.19 main 関数の基本形

```
1 def main():  
2     print("run")  
3  
4 if __name__ == "__main__":
```

この形にしておくと、モジュールとして読み込んだときには関数だけを使え、直接実行したときだけ本体が動く。後半の解析スクリプトでも、この形を基本として採用すると整理しやすい。

4.17 エラーは失敗ではなく情報である

初心者のうちは、エラーメッセージが出るとそこで止まってしまいがちである。しかし、トレースバックは「どこで何が起きたか」を教えてくれる重要な情報である。

たとえば `FileNotFoundError` ならファイルパスが違うかもしれないし、`ValueError` なら想定した形式でデータを読めていないかもしれない。大切なのは、英語の文章全体を恐れるのではなく、例外名と最後の数行を見ることである。

解析では、最初から一度もエラーを出さずに進む方が珍しい。問題はエラーが出ることでなく、エラーを無視したり、意味を確かめずに場当たりの直したりすることである。例外名、発生行、直前に読んでいたデータの形を確認する習慣を付けておくと、修正の質が安定する。

4.18 docstring とヘルプ

関数やモジュールの役割を後から思い出せるようにするには、短い説明を書いておくとうい。Python では、関数定義の直後に文字列を書くと docstring になる。

リスト 4.20 docstring の例

```
1 def read_table(path):
2     """空白区切りの表を読み、各行を辞書として返す。"""
3     ...
```

`help(read_table)` を実行すると、この説明を後から確認できる。コメントと同じように見えるが、実行時にも参照できる点が異なる。解析コードが増えてくると、どの関数が何を返すのかを書いておくことの価値が大きくなる。

4.19 名前付けと PEP 8

Python には PEP 8 という代表的なスタイルガイドがある。すべてを最初から覚える必要はないが、少なくとも変数名、関数名、空白の入れ方をそろえるだけでコードはかなり読みやすくなる。

リスト 4.21 PEP 8 を意識した名前付けの例

```
1 MAX_EVENTS = 1000
2
```



```

3 def read_event_table(path):
4     total_count = 0
5     ...

```

一般には、変数名や関数名は `snake_case`、定数は大文字の `UPPER_CASE` にすることが多い。さらに、演算子の前後に空白を入れる、1 行を詰め込みすぎない、といった基本だけでも効果は大きい。スタイルの統一は飾りではなく、バグを見つけやすくするための道具だと考える方が実践的である。

4.20 マジックナンバーを避ける

コード中に 250 や 3600 のような数値が突然現れると、その値が何を表すのかを読む側が推測しなければならない。このような、説明なしに裸で現れる数値をマジックナンバーと呼ぶ。

リスト 4.22 マジックナンバーを避ける例

```

1 NS_PER_SECOND = 1_000_000_000
2 COINCIDENCE_WINDOW_NS = 250
3
4 dt_ns = second_diff * NS_PER_SECOND + ns_diff
5 is_coincident = abs(dt_ns) <= COINCIDENCE_WINDOW_NS

```

250 をそのまま書くより、`COINCIDENCE_WINDOW_NS` と書いた方が意味が明確である。3600 なら `SECONDS_PER_HOUR`、86400 なら `SECONDS_PER_DAY` といった具合に、意味を名前へ出した方が読みやすい。

ただし、どんな値でもコード内定数にすればよいわけではない。物理定数や単位換算のように定義として固定したい値はコード内の定数に向いている。一方で、入力パス、出力先、閾値、利用する検出器番号のように実行条件として変わり得る値は、コマンドライン引数や設定ファイルへ出した方がよい。値の置き場所そのものが設計である、という意識を持つことが重要である。

4.21 例外処理

エラーは読むべき情報だが、予想される失敗を自分で受け止めたい場面もある。例えば、1 行だけ形式が壊れているデータを見つけて記録し、解析全体は続けたいことがある。そのときに使うのが `try` と `except` である。

リスト 4.23 `try` と `except` の例

```

1 try:
2     value = float(text)
3 except ValueError:
4     print("数値へ変換できませんでした")

```

ただし、何でも `except:` で受けてしまうと、本来止まるべきバグまで隠してしまう。どの例外を受けるのかを明示し、何を記録して、何を諦めて、何を継続するのかを自分で決めることが重要である。

第 5 章 入出力とファイル

この章では、Python でファイルを開き、複数のファイルを集め、1 行ずつ読みながら値へ変換する基本を扱う。ここで重要なのは、特定のデータ形式に慣れることではなく、入力を見て必要な情報を取り出す流れを理解することである。

5.1 open と with

Python でテキストファイルを読むときの基本は `open` である。さらに `with` を組み合わせると、読み終わった後に自動でファイルが閉じられる。

リスト 5.1 テキストファイルを 1 行ずつ読む基本形

```
1 with open(path, "r", encoding="utf-8") as f:
2     for line in f:
3         print(line.rstrip())
```

`read` でまとめて読む方法もあるが、大きなファイルでは 1 行ずつ読む方が安全である。まずはこの形に慣れておくとよい。

このとき、`line` には改行文字が含まれている点にも注意したい。表示だけなら `rstrip` で末尾を落とせば十分だが、区切り文字を扱うときには改行の有無が結果に影響することがある。テキスト処理では、見えていない文字を意識することが重要である。

5.2 pathlib でパスを扱う

ファイルパスを単なる文字列として扱うこともできるが、Python では `pathlib.Path` を使うと見通しが良くなる。

リスト 5.2 pathlib の例

```
1 from pathlib import Path
2
3 root = Path("data")
4 path = root / "2026" / "04" / "sample.txt"
5
6 print(path.exists())
7 print(path.name)
```

`/` 演算子でパスをつなげられるため、文字列連結より読みやすい。ファイル名だけ取りたい、親ディレクトリへ戻りたい、拡張子を変えたい、といった操作も自然に書ける。解析コードでは、パス操作を文字列で済ませるより安全なことが多い。

リスト 5.3 pathlib で出力先を作る例

```
1 outdir = Path("output")
2 outdir.mkdir(parents=True, exist_ok=True)
3 outfile = outdir / "summary.txt"
```

ディレクトリがなければ作る、すでにあってもエラーにしない、という処理は解析コードで非常によく使う。こうした定型操作を安全に書ける点でも pathlib は便利である。

5.3 ファイルオブジェクトのメソッド

ファイルを開いた後には、read、readline、write などのメソッドが使える。Python 公式チュートリアルでも強調されているように、何をどの粒度で読むかでコードの性格が変わる。

リスト 5.4 file object の基本操作

```
1 with open(path, "r", encoding="utf-8") as f:
2     first_line = f.readline()
3
4 with open("out.txt", "w", encoding="utf-8") as f:
5     f.write("hello\n")
```

readline は 1 行だけ確認したいとき、write は整形済み文字列をそのまま出したいときに便利である。大きな入力全体を一気に読む前に、最初の数行だけ確認するという使い方もよく行う。

5.4 複数のファイルを集める

ログや測定結果が複数ファイルに分かれている場合は、まず対象ファイルの一覧を作る必要がある。Python では glob.glob がよく使われる。

リスト 5.5 glob によるファイル探索

```
1 import glob
2
3 paths = sorted(glob.glob("data/**/*.txt", recursive=True))
```

glob.glob はシェルのワイルドカードに近い感覚で使える。複数ファイルを扱う処理では、最初に見つけた件数や先頭数件のパスを表示して確認する習慣を付けておくとよい。

また、順序が重要な処理では sorted を付ける意味も大きい。ファイル探索の結果が環境依存の順序で返ると、処理結果やログの見え方が毎回変わり、デバッグしにくくなる。時系列や連番を扱う場面では、最初に順序を固定する方が安全である。

5.5 1 行の文字列を値へ変換する

多くの表形式データは、1 行の中に複数の列が並ぶ形で保存されている。テキストとして読んだだけでは、まだ単なる文字列である。そこから列に分け、整数や浮動小数点数へ変換して初めて解析に使える。

リスト 5.6 空白区切りの 1 行を分解する例

```
1 line = "17_2026-04-04_12:34:56_15.2_OK"
2 columns = line.split()
3
4 record_id = int(columns[0])
5 date_text = columns[1]
6 time_text = columns[2]
7 value = float(columns[3])
8 status = columns[4]
```

ここで重要なのは、どの列をどの型で持つかを自分で決めることである。後で pandas を使う場合でも、この感覚を持っていると列の意味を見失いにくい。

文字列をどの段階で数値へ変換するかも設計の一部である。すぐ数値化してよい列もあれば、元の文字列表現を残しておいた方が確認しやすい列もある。読みやすい解析コードでは、どこでどの変換を行ったかが追えるようになっている。

5.6 ファイルへ書き出す

解析では、読むだけでなく結果を書き出す場面も多い。最も単純には、テキストとして 1 行ずつ書いていけばよい。

リスト 5.7 テキストを書き出す最小例

```
1 lines = ["run_id_count_mean", "1_120_3.42", "2_118_3.37"]
2
3 with open("summary.txt", "w", encoding="utf-8") as f:
4     for line in lines:
5         f.write(line + "\n")
```

"w" は新しく書くモード、"a" は追記モードである。既存ファイルを上書きしてよいのか、別名で保存すべきかは、実験ノートやレポート再現性にも関わる。出力ファイル名の設計は、解析の一部として考えるべきである。

5.7 文字列整形

結果を画面やファイルへ出すときには、数値を読みやすく整えることも大切である。Python では、f-string が最もよく使われる。

リスト 5.8 f-string の例

```
1 mean = 3.14159
2 sigma = 0.27182
3
4 text = f"mean={mean:.3f}, sigma={sigma:.3f}"
5 print(text)
```

`:.3f` は小数点以下 3 桁で表示するという意味である。解析結果をレポートへ貼る前に、このような整形で見やすくしておくと、図や本文との対応が取りやすい。

Python 公式チュートリアルでは、f-string のほかに `str.format` や手動整形も紹介されている。現在の実務では f-string が最も使いやすいが、古いコードを読むと別の書き方が出てくこともある。複数の流儀があることは知っておきたい。

5.8 JSON

結果や設定を構造化して保存したいときは、JSON が便利である。テキストとして読めて、Python では標準ライブラリ `json` ですぐ扱える。

リスト 5.9 JSON の保存と読み込み

```
1 import json
2
3 config = {"threshold": 3.5, "bins": 40}
4
5 with open("config.json", "w", encoding="utf-8") as f:
6     json.dump(config, f, indent=2)
7
8 with open("config.json", "r", encoding="utf-8") as f:
9     loaded_config = json.load(f)
```

大きな表形式データの本体を JSON で持つのは効率が悪いことも多いが、設定や要約結果を保存するには便利である。人間が読める形式と、Python が簡単に読める形式を両立しやすい。

5.9 Pickle / PKL

Python オブジェクトをそのまま保存したいときは、`pickle` もよく使われる。拡張子は慣習的に `.pkl` を使うことが多い。辞書、リスト、NumPy 配列、学習済みモデル、中間集計結果などを素早く退避するには便利である。

リスト 5.10 pickle の保存と読み込み

```
1 import pickle
2
3 result = {"mean": 3.14, "sigma": 0.27, "values": [1.0, 2.0, 3.0]}
4
```

```

5 with open("result.pkl", "wb") as f:
6     pickle.dump(result, f)
7
8 with open("result.pkl", "rb") as f:
9     loaded_result = pickle.load(f)

```

"wb" と "rb" の b はバイナリモードを意味する。JSON と違って人間には読めないが、Python では構造を保ったまま簡単に保存できる。ただし、pickle は Python 専用であり、他言語との受け渡しには向かない。また、信頼できない .pkl ファイルを load してはならない。共有用の正式な保存形式というより、自分の解析途中結果を手早く再利用するためのローカル形式と考える方が安全である。

5.10 日付と時刻を文字列のままにしない

日付や時刻を含むデータでは、文字列のまま扱うより、比較や計算がしやすい形へ変換した方がよい。標準ライブラリの datetime は、そのための基本的な道具である。

リスト 5.11 datetime を使って時刻を扱う例

```

1 from datetime import datetime
2
3 text = "2026-04-04_12:34:56.123456"
4 dt = datetime.strptime(text, "%Y-%m-%d_%H:%M:%S.%f")
5 unix_time = dt.timestamp()

```

秒単位の差を見たいのか、より細かい時間差を見たいのかによって、どの単位で値を持つかは変わる。ただし、どの表現を選んでも、一貫して使うことが大切である。

datetime は便利だが、何でも datetime オブジェクトのまま持てばよいわけではない。差分計算を大量に繰り返すなら、途中で秒やナノ秒の数値へ変換した方が扱いやすいことも多い。人間に読みやすい表現と計算しやすい表現を分けて考えることが重要である。

さらに、タイムゾーンが関わるデータでは、見かけの時刻だけを比較すると誤解が生じる。研究用のログや観測データでは UTC を使うことが多いが、どの基準時刻を採用しているかを最初に確認する習慣を付けたい。

5.11 保存形式について: ベストプラクティスへのステップ

データをファイルに保存・読み出しする際、多くの初学者は何も考えずに .csv (CSV 形式) を使い続ける。しかし、データが数百万行に達すると、処理時間の大半が「ファイルの準備」に消えるようになる。

5.11.1 Step 1: テキスト形式 (CSV, JSON) の限界

テキスト形式は文字コード (例: UTF-8) を使って人間がそのまま読める形で保存する。CSV/TSV は非常に汎用性が高く、C++ や R、Excel など、どの言語でも読めるのが強みで

ある。

問題点: コンピュータはディスク上の文字列（例：「1.2345」という 6 文字の羅列）を、計算に使える実数値にいちいち変換（パース）しなければならない。データ量が数 GB に及ぶと、この「文字列から実数への変換処理」だけで何十分も待たされることになる。

5.11.2 Step 2: バイナリ形式への移行と、どれを選ぶかの判断基準

メモリ上の数値を人間には読めない生のバイト列としてそのままファイルに記録する「バイナリ形式」を使えば、読み込み時間は一瞬（ディスクの転送速度の限界）になる。

しかし、バイナリ形式にはいくつかデファクトスタンダードがあり、目的に合わせて選ばなければならない。

- **NPY / NPZ (NumPy 標準):** NumPy だけで完結する小さな解析なら最も手軽。ただし、Python・NumPy 以外（C++や R など）からは読みづらく使い捨てになりがちである。
- **Pickle / PKL:** Python オブジェクトをそのまま保存できるため、中間結果や学習済みモデルの一時保存に便利である。ただし、Python 以外ではほぼ使えず、信頼できないファイルを読み込んでではない。共同研究の交換形式というより、ローカルな作業用キャッシュとして使うべき形式である。
- **HDF5:** 大規模な数値データをディレクトリの階層構造のように管理する形式。C/C++ (Geant4 など)、Fortran、R 等さまざまな言語の公式ライブラリが存在する。他のグループや別言語のプログラムにデータを渡す場合は、HDF5 を選ぶのが最も安全で汎用性が高い。
- **Parquet:** 列指向のバイナリフォーマット。後述する Pandas や Polars などの DataFrame をそのまま保存でき、特定の列だけを高速に読み込むことが得意。Hadoop や Spark などのビッグデータエコシステムでも標準的である。分析チーム内での巨大データのやり取りにはこれが現在の最強の選択肢である。

解析の初期段階ではデータの中身を目視確認するために CSV を使うことが多いが、大量のイベントデータを何回も読み直す段階になったら、**必ず最初に Parquet や HDF5 への変換スクリプトを書いてバイナリ化することが全体を高速化する第一歩である**。`.pkl` はその補助として、途中結果や作業用キャッシュを手元で素早く保存する用途に向いている。

第 6 章 NumPy による数値計算

6.1 なぜ NumPy を使うのか

Python のリストは柔軟だが、数値配列として大量の値を処理するには向いていないことがある。NumPy は、同じ型の数値を連続した配列として持ち、まとめて計算できるようにするライブラリである。

リスト 6.1 NumPy 配列の例

```
1 import numpy as np
2
3 times = np.array([10, 15, 23, 41], dtype=np.int64)
4 deltas = times[1:] - times[:-1]
```

このような差分計算は、Python の for ループでも書けるが、NumPy では配列全体に対してまとめて書ける。後の図 10.1 に示すように、この差は速度にも大きく効く。

ここで重要なのは、NumPy が単に「速いリスト」ではないという点である。NumPy 配列は、同じ型の数値をまとめて持ち、一つの式で大量の要素を処理するための道具である。したがって、設計の発想も「1 要素ずつ何をするか」から「配列全体にどのような変換をかけるか」へ変わる。

6.2 NumPy が速くなりやすい理由

NumPy が速くなりやすい理由は、単に「ライブラリだから」ではない。主な理由は三つある。

1. 数値を連続したメモリにまとめて持てること
2. Python レベルのループを減らせること
3. 内部で低レベルに最適化された処理を使えること

特に重要なのは、Python で 1 要素ずつ処理する回数が減ることである。大きなデータでは、この差がそのまま処理時間の差になる。

もう一つの利点は、式がそのまま数学的な記述に近づくことである。たとえば差分、閾値判定、正規化、平均、分散といった操作は、配列演算で書いた方が意図が明確になることが多い。読みやすさと速度が同時に改善する場面は少なくない。

6.3 よく使う操作

解析でよく使う NumPy の操作には、次のようなものがある。

- フィルタ: 条件に合う要素だけを取り出す
- 差分: 隣り合う要素の差を取る
- ソート: 時刻順に並べる
- ヒストグラム: 分布を数える

リスト 6.2 条件抽出と差分

```
1 selected = times[times > 20]
2 deltas = np.diff(times)
```

ブール配列によるフィルタは NumPy の基本である。条件式の結果が True と False の配列になり、それをそのまま添字として使える。最初は不思議に見えるが、この書き方に慣れると、複数条件の組合せも簡潔に書けるようになる。

6.4 形とブロードキャスト

NumPy では、配列の shape を意識することが重要である。1 次元配列なのか、行列なのか、列ベクトルとして扱いたいのかで、同じ演算でも結果が変わる。

リスト 6.3 shape とブロードキャストの例

```
1 x = np.array([1.0, 2.0, 3.0])
2 y = np.array([10.0, 20.0, 30.0])
3
4 z = x + y
5 matrix = x[:, None] + y[None, :]
```

後半の matrix では、1 次元配列を軸付きで広げることで、3 行 3 列の和の表を作っている。こうした自動拡張の規則をブロードキャストという。最初は少し難しいが、二つの配列の全組合せを調べるような場面で非常に強力である。

リスト 6.4 shape とブロードキャストの結果例

```
z =
[11. 22. 33.]

matrix =
[[11. 21. 31.]
 [12. 22. 32.]
 [13. 23. 33.]]
```

この結果から、z は要素ごとの和、matrix は全組合せの表であることが分かる。便利では

あるが、配列サイズが大きいと一気にメモリを消費するので、実データでは常に形と大きさを見積もる必要がある。

6.5 統計量とヒストグラム

NumPy には、平均、分散、最小値、最大値、ヒストグラムなど、解析の基本となる集計関数がそろっている。

リスト 6.5 統計量とヒストグラムの例

```
1 mean = values.mean()
2 std = values.std(ddof=0)
3 hist, edges = np.histogram(values, bins=50)
```

`np.histogram` は図を描かずに分布だけを数えたいときに便利である。ビン境界 `edges` も同時に返すため、後で自分で曲線を重ねたり、複数データを比較したりしやすい。可視化と数え上げを分けて考えると、処理の見通しが良くなる。

リスト 6.6 統計量の出力例

```
mean = 2.31
std = 0.84
hist.shape = (50,)
edges.shape = (51,)
```

平均や標準偏差だけでは分布の形は分からず、ヒストグラムだけでは条件間比較がしにくい。数値要約と分布表示を組み合わせる読むことが、データ解析では基本になる。

6.6 配列処理の注意

NumPy は便利だが、配列をむやみにコピーすると逆に重くなることがある。さらに、型を適切に選ばないとメモリを無駄に使う。

たとえば、識別子や時刻差なら `int64` が自然だが、カテゴリ番号が小さい範囲なら `int16` や `int32` でも十分なことがある。大規模データでは、この積み重ねが効いてくる。

また、配列の一部を取り出したつもりでも、実際には元配列の参照になっている場合とコピーになっている場合がある。この違いを理解せずに値を書き換えると、意図しない副作用や余分なメモリ使用を招く。NumPy を使い始めたら、`view` と `copy` の違いも徐々に意識したい。

浮動小数点数の丸め誤差にも注意が必要である。理論上は等しいはずの値でも、計算手順によって最後の数桁がずれることがある。そのため、`a == b` のような直接比較より、許容誤差を設けた比較を使う方が安全なことが多い。

6.7 なぜ NumPy を使うのか：リストの限界

Python の標準リストは非常に柔軟であり、どんな型のものでも放り込める。しかしいざ大量の数値を計算しようとする、この「なんでも入る」という性質が最大のネックとなる。

6.7.1 Step 1: Python リストの素朴なアプローチ

リスト 6.7 リストを用いた数値計算の遅さ

```
1 # 100万個のリストを作る
2 py_list = list(range(1000000))
3
4 # すべての要素を2倍にする。forループと内包表記が必要
5 doubled_list = [x * 2 for x in py_list]
```

一見普通に動くが、Python のリストは「あちこちに散らばっている多様なサイズの箱へのポインタ（地図）」の集まりである。CPU は一つの値を計算するたびに「この値は何型か？」「どこに置かれているか？」を確認しなければならず、絶望的に遅い。

6.7.2 Step 2: NumPy のベクタライズ化

これに対し NumPy デイメンショナル配列 (ndarray) は、同じ型の数値 (int64 や float64 など) をメモリ上に「すき間なく一列に並べた団地」として構成される。型確認が不要で、C 言語レベルの一括演算回路が直結する。

リスト 6.8 NumPy への変換と一括計算

```
1 import numpy as np
2
3 # 1. リストから NumPy アレイへの変換（メモリ上にきれいに並べ直す）
4 np_array = np.array(py_list)
5
6 # 2. アレイはベクトル演算が可能（for文を書かなくても全要素を一気に計算できる）
7 doubled_array = np_array * 2
8
9 # 3. アレイからリストへ戻すことも可能だが、重いので通常は最後だけにする
10 back_to_list = doubled_array.tolist()
```

6.8 Pandas DataFrame との変換

後述する Pandas DataFrame は、この NumPy 配列を「列」として束ねた拡張版である。分析の途中で NumPy 独自の計算関数を使いたい場合、DataFrame を NumPy に戻すことができる。

リスト 6.9 Pandas の DataFrame・Series と NumPy アレイの相互変換

```
1 import pandas as pd
2
3 # NumPy アレイから pandas の DataFrame (二次元) に変換
4 df = pd.DataFrame(np_array, columns=["Value"])
5
6 # Pandas のデータから NumPy アレイに戻す
7 # \code{.to_numpy()} は背後で持っているC言語配列を直接取り出すため一瞬で終わる
8 values_array = df["Value"].to_numpy()
```


第7章 pandas による大規模データ解析

7.1 DataFrame とは何か

pandas の中心にある DataFrame は、列名付きの表である。観測ログや測定結果のように「行が記録、列が属性」という形のデータには特に向いている。

リスト 7.1 pandas の最小例

```
1 import pandas as pd
2
3 df = pd.read_csv(path, sep=r"\s+", comment="#", header=None)
```

DataFrame の利点は、列ごとに意味を持たせながら、まとめて処理を書けることにある。

DataFrame を使うと、「何列目か」を覚える必要が減り、列名を通して処理の意味が見えやすくなる。特に、ファイル読み込み、列変換、フィルタ、集計、書き出しまでを一つの流れで扱える点は大きい。解析の途中で確認用の表を少し表示したいときにも便利である。

DataFrame の基本操作は、Python 公式チュートリアル「データ構造」章に相当する考え方を、表形式へ拡張したものだと考えると理解しやすい。リストや辞書を知っていると、列名によるアクセスや行集合に対するフィルタの意味が見えやすい。

リスト 7.2 DataFrame の中身を確認する例

```
1 print(df.head())
2 print(df.dtypes)
```

大きなデータを読むときほど、最初に head() と dtypes を確認する価値がある。列名は正しいか、型は意図通りか、欠損はないかを早い段階で見るだけで、多くのバグを防げる。

7.2 列選択、並べ替え、集約

pandas を使い始めたら、まず列選択と並べ替えを自然に書けるようになりたい。

リスト 7.3 列選択と並べ替えの例

```
1 selected = df[["date", "value"]]
2 sorted_df = df.sort_values("value")
3 top10 = sorted_df.tail(10)
```

ここで重要なのは、行ごとの処理より先に、表全体の形を整えることである。必要列だけを取り出し、順序をそろえ、不要な行を除く。この前処理がきれいだと、その後の集計も読みやすくなる。

リスト 7.4 groupby の例

```
1 summary = df.groupby("category")["value"].mean()
```

groupby は、カテゴリごとに平均や件数を出したいときの基本である。研究データでは、検出器ごと、条件ごと、日ごとのように、同じ操作を複数群へ適用したい場面が多い。手書きのループよりも、列の意味を保ったまま集約できる点が pandas の強みである。

リスト 7.5 groupby 集約結果の例

```
category
A 3.41
B 3.08
C 2.95
Name: value, dtype: float64
```

この結果例では、カテゴリごとの平均値をすぐ比較できる。必要に応じて件数や標準偏差も一緒に出せば、単に平均が違うのか、ばらつきやサンプル数も違うのかを確認できる。

7.3 pandas が速くなりやすい理由

pandas が速くなりやすいのは、列単位でデータを処理し、内部でベクトル化された操作を使えるからである。Python の辞書リストを 1 行ずつ回して書くより、列としてまとめて扱う方が速く、コードも短くなりやすい。

ただし、すべての場面で pandas が最適とは限らない。行ごとに複雑な条件分岐を大量に行う場合や、データ構造自体を細かく制御したい場合には、NumPy や自作構造の方が向くこともある。

したがって、「とりあえず pandas を使う」よりも、「列として扱える問題かどうか」を考える方が重要である。表として整理できるなら pandas は強力だが、近傍探索や状態遷移のように行どうしの関係が複雑になると、別の設計の方が自然なこともある。

7.4 大きなテキストを読む工夫

read_csv を使うときは、必要な列だけ読む、型を指定する、コメント行を飛ばす、といった工夫が重要である。

リスト 7.6 必要な列だけを読む例

```
1 df = pd.read_csv(
2     path,
3     compression="gzip",
4     sep=r"\s+",
5     comment="#",
6     header=None,
7     usecols=[0, 1, 2, 3],
8     names=["id", "date", "time", "value"],
```


このように列を絞るだけでも、読み込み時間とメモリ消費はかなり変わる。

さらに、dtype を指定しておく、読み込み後の型変換を減らせる。列数や行数が大きいデータでは、どの列が本当に必要かを考えるだけで、後続の処理全体が軽くなる。読み込みは解析の最初の一步だが、同時に最適化の出発点でもある。

7.5 CSV を毎回読み直さず Parquet へ変換する

前章で述べたように、巨大なテキストファイルを何度も read_csv するのは無駄が大きい。列構造が固まったら、一度だけ CSV から DataFrame を作り、その後は Parquet として保存して再利用する方が実務的である。

リスト 7.7 CSV から Parquet へ一度だけ変換する例

```
1 raw_df = pd.read_csv(
2     "events.csv.gz",
3     sep=r"\s+",
4     comment="#",
5     header=None,
6     names=["id", "date", "time", "value"],
7 )
8 raw_df.to_parquet("events.parquet")
9
10 df = pd.read_parquet("events.parquet", columns=["id", "time", "value"])
```

read_parquet なら、文字列のパースを毎回やり直さずに済み、必要列だけを読むこともできる。日々の解析では、この「最初に 1 回だけテキストを整理して、以後は Parquet を読む」という流れを作るだけで、待ち時間が大きく減る。なお、Parquet の入出力には pyarrow などのバックエンドが必要なことが多い。

リスト 7.8 列演算とフィルタの例

```
1 df["ratio"] = df["value"] / df["value"].mean()
2 selected = df[df["ratio"] > 1.5]
```

このように、列同士の演算をそのまま式として書ける点が pandas の強みである。行ごとの for ループを書く前に、列単位で表現できないかを考える習慣を持つとよい。

7.6 欠損値と型の扱い

現実のデータでは、すべての行がきれいに埋まっているとは限らない。空欄や壊れた値が混ざると、pandas では NaN として表現されることが多い。

リスト 7.9 欠損値の確認と除外

```
1 missing_mask = df["value"].isna()
```

```
2 clean_df = df.dropna(subset=["value"])
```

欠損値をどう扱うかは、単なる実装問題ではなく解析方針そのものである。除外してよいのか、別扱いすべきか、補完してよいのかを、物理的な意味と合わせて考える必要がある。型の自動推定に任せきりにせず、重要な列の型は自分で確認したい。

7.7 結合と時系列処理

複数の表をまとめたり、時刻順に並べ替えたりする処理も pandas の重要な役割である。

リスト 7.10 連結と並べ替えの例

```
1 combined = pd.concat([df1, df2], ignore_index=True)
2 combined = combined.sort_values("time")
```

複数ファイルを月単位や日単位で読んで、最後にまとめるという流れは現実の解析でよく現れる。どの段階で結合するか、結合後にどの列で並べ替えるかを明確にしておく、ファイルまたぎの不具合を減らしやすい。

リスト 7.11 時刻列を datetime へ変換する例

```
1 combined["time"] = pd.to_datetime(combined["date"] + "_" + combined["time"])
2 combined = combined.sort_values("time")
```

文字列のままでは、見た目には時刻に見えても、差分計算や順序比較には向かないことがある。早い段階で型を変えておくと、その後の処理が自然になる。

7.8 DataFrame を TeX で出力する

解析結果として集約した表などを LaTeX のレポートに直接貼り付けたい場合は、手作業で打ち直すのではなく `to_latex()` を用いると便利である。

リスト 7.12 DataFrame の TeX 形式出力

```
1 # pandas のバージョンにより引数の推奨が変わる場合があるが、一般的な利用法
2 tex_table = summary.to_latex(
3     caption="カテゴリごとの平均値",
4     label="tab:category_mean",
5     float_format="%.3f" # 小数点以下3桁に整える
6 )
7 print(tex_table)
```

これを出力したテキストファイルに保存し、本ドキュメント側で `include` することで、解析スクリプトからレポートまでのパイプラインが完成する。

7.9 Pandas の行処理と並列化：論理的改善ステップ

Pandas の DataFrame は数百万行のデータを保持できるが、各行に対して複雑な文字列処理や条件判定を加えようとした時、コードの書き方一つで実行時間が数百分から数秒へと劇的に変わる。

7.9.1 Step 1: 絶対に避けるべき for ループ (iterrows の限界)

Python 初学者が最も陥りやすい罠が、「DataFrame の行を 1 つずつ取り出して処理する」という直感的な書き方である。

リスト 7.13 iterrows による行単位のループ処理

```
1 def heavy_computation(val):
2     return val ** 2
3
4 # 絶望的に遅い：1行ずつPython空間でオブジェクトが作られ、評価される
5 results = []
6 for index, row in df.iterrows():
7     results.append(heavy_computation(row["value"]))
8 df["value_sq"] = results
```

このコードは 10 万行を超えたあたりから全く終わらなくなる。Pandas がデータフレームとしてデータを一括管理している利点を全て打ち消し、Python の重い関数呼び出しを数百万回繰り返しているからである。

7.9.2 Step 2: Pandas ネイティブな apply() への移行

行ごとに処理を適用したい場合、Pandas が用意している apply() メソッドを使うのが正しい作法である。内部のループ処理が C 言語ベースで最適化されているため、for ループより劇的に高速になる。

リスト 7.14 apply を用いた行単位処理

```
1 # for 文を書かずに、関数そのものを列に適用（マッピング）する
2 df["value_sq"] = df.apply(lambda row: heavy_computation(row["value"]), axis=1)
```

ただし、axis=1 を付けた apply() も各行に対して Python 関数を呼ぶため、列演算ほど速いわけではない。上の例のように単に二乗したいだけなら、実際には次のように列演算で書く方が自然で速い。

```
df["value_sq"] = df["value"] ** 2
```

apply() は「列演算だけでは書きにくい、まだ pandas の文脈の中で処理したい」場合の中間手段として理解の方が正確である。まず列演算、次に apply()、それでも重ければ並列化、という順で判断すると迷いにくい。

7.9.3 Step 3: 並列化とボトルネックの判断 (Pandarallel)

しかし、`apply()` はどれだけ行数があっても「CPU の 1 つのコア」しか使わない（シングルスレッド）。本当に計算が重い処理の場合、`apply` を使っても処理が長引いてしまう。ここで初めて、マルチコアを用いた「並列化（マルチプロセッシング）」が選択肢に入る。

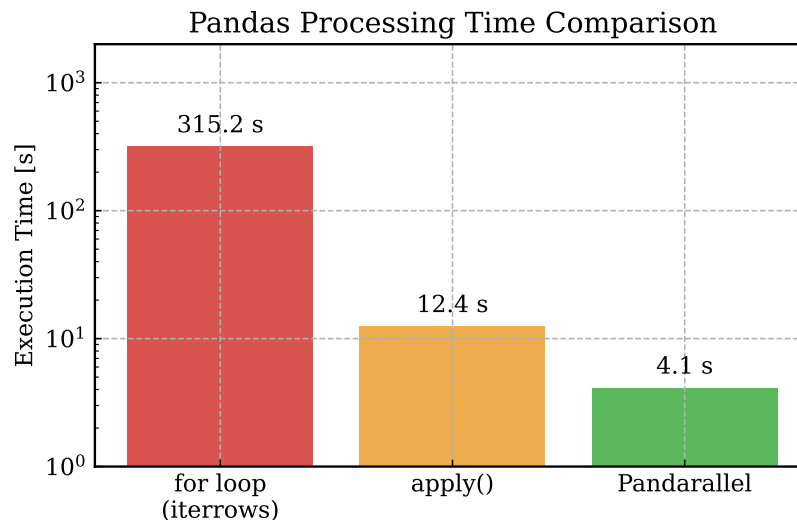


図 7.1 同じ重い計算処理を、Python のループ、`apply` 関数、`Pandarallel` で行った際の処理時間の激変の様子。

リスト 7.15 `Pandarallel` を用いた最強の並列化

```
1 from pandarallel import pandarallel
2
3 # 並列環境の初期化。プログレスバーを表示する設定をオンにしている
4 pandarallel.initialize(progress_bar=True)
5
6 # apply を parallel_apply に置き換えるだけで、全CPUコアに処理が分散される
7 df["value_sq"] = df.parallel_apply(lambda row: heavy_computation(row["value"]),
    axis=1)
```

図 7.1 を見ると明らかなように、手段の選定によって処理時間は 2 桁変わる。

`pandarallel` は Linux 系環境での利用が比較的安定している一方で、ネイティブの Windows 環境では設定や挙動で悩みやすいことがある。Windows で本書の流れを再現するなら、前章で触れた WSL を使う方が混乱しにくい。

並列化の罠（判断基準）： 並列化は魔法の杖ではない。`pandarallel` は背後で「データを複数のプロセス（別のメモリ空間）にコピーして分配する」ため、通信オーバーヘッド（コピーの手間）がかかる。「単純な足し算」などのすぐに終わる軽い処理を `parallel_apply` にかけると、計算時間よりもコピー時間の方が長くなり、かえって遅くなる。「CPU1 コアでは計算に時間がかかりすぎる重い処理」でのみ並列化を採用するという見極めが必要である。

7.10 さらに大規模なデータへの対応 (Polars など)

pandas は強力だが、全データを一度にメモリ (RAM) に載せる必要がある設計上、マシンのメモリを超えるようなテラバイト級のデータを処理させるとクラッシュしてしまう。また、内部的なループの非効率さも残っている。近年では、言語の内部実装に Rust を用いるなどで劇的な高速化とメモリ効率の改善を図った Polars 等のライブラリの利用も急速に広がっている。もし数千万行に及ぶ処理で pandas の限界を感じた場合は、Polars への移行を検討するのも有効なアプローチである。

第 8 章 Matplotlib による可視化

8.1 図は結果そのものである

図はレポートの飾りではない。ヒストグラムや散布図は、計算結果の妥当性を自分で確かめるための道具でもある。数値だけを並べるより、まず図にした方が異常に気付きやすいことは多い。

例えば、平均値や標準偏差が妥当に見えても、分布の裾が長い、二峰性がある、外れ値が混ざっている、といった特徴は図にしないと分かりにくい。可視化は最後の仕上げではなく、途中の確認作業としても重要である。

8.2 最小限のヒストグラム

リスト 8.1 ヒストグラムの最小例

```
1 import matplotlib.pyplot as plt
2
3 plt.hist(values, bins=40)
4 plt.xlabel("value")
5 plt.ylabel("count")
6 plt.savefig("histogram.pdf")
```

解析では PDF 出力が便利である。拡大しても崩れにくく、LaTeX レポートへそのまま入れやすいからである。図 8.1 は、本章と次章で扱う基本的な図とフィットの例である。

`plt.show` による対話的な確認は便利だが、再現可能な解析では `savefig` でファイルへ残す方が重要になる。図を保存するときに同じファイル名へ上書きするのか、条件ごとに別名で保存するのかも、早い段階で決めておくと混乱しにくい。

8.3 オブジェクト指向 API で描く

Matplotlib には `plt.hist(...)` のような簡単な書き方と、`Figure` と `Axes` を明示的に扱う書き方がある。図が複雑になるほど、後者の方が整理しやすい。

リスト 8.2 `Axes` を使う描画の例

```
1 fig, ax = plt.subplots(figsize=(6, 4))
2 ax.hist(values, bins=40)
3 ax.set_xlabel("value")
4 ax.set_ylabel("count")
```

```
5 fig.savefig("histogram.pdf")
```

ax に対して操作を書く形にしておくと、複数図や複数パネルへ発展しやすい。教材やレポート用のコードでは、この書き方に慣れておく価値が高い。

また、Figure と Axes を区別して考えると、図全体に効く設定と各パネルにだけ効く設定を整理しやすい。タイトルや余白は図全体、軸ラベルや凡例は各パネル、というように役割を分けて考えると見通しが良い。

8.4 見た目を整える

図を読みやすくするには、少なくとも次の点を意識する。

- 軸ラベルに量と単位を書く
- タイトルを簡潔に付ける
- 余白を調整する
- 色と線幅を統一する
- レポート全体で表記を揃える

色やフォント、線幅をばらばらにすると、個々の図は読めてもレポート全体として落ち着かない。逆に、軸ラベルの位置、単位の書き方、凡例のスタイルをそろえるだけで、見た目の質は大きく上がる。可視化では、派手さより統一感の方が効くことが多い。

8.5 rcParams で見た目をそろえる

Matplotlib では、図のフォントや線幅などの既定値の多くを rcParams という辞書オブジェクトで一元管理している。図を描くたびに同じ設定を繰り返すより、作図スクリプトの冒頭で一度だけこの辞書を更新するのが基本である。

リスト 8.3 rcParams をまとめて設定する基本例

```
1 import matplotlib as mpl
2
3 mpl.rcParams.update(
4     {
5         "font.family": "serif",
6         "mathtext.fontset": "cm",
7         "font.size": 12,
8         "axes.grid": True,
9         "grid.linestyle": "--",
10        "xtick.direction": "in",
11        "ytick.direction": "in",
12        "axes.linewidth": 1.2,
13        "legend.frameon": False,
14    }
```


8.6 設定ファイル (.mplstyle) で見た目を共通化する

しかし、解析テーマごとに作図スクリプトが何個も増えていくと、すべての Python ファイルの先頭に上記の長い辞書をコピーして回るのは非常に冗長である。Matplotlib には、この設定内容を一つのプレーンテキストに切り出して一括で読み込む、よりスマートな仕組みが存在する。

ご自身の Python スクリプトを置いている作業ディレクトリに `my_style.mplstyle` という名前で以下のようなテキストファイルを作成する。

リスト 8.4 `my_style.mplstyle` の例

```
font.family: serif
mathtext.fontset: cm
font.size: 12
axes.grid: True
grid.linestyle: --
xtick.direction: in
ytick.direction: in
axes.linewidth: 1.2
legend.frameon: False
```

そして、各解析スクリプトの先頭でこれを一度だけ呼び出せばよい。本テキストに含まれる図も、すべてこの設定ファイル（スタイルシート）を読み込んで生成されている。

リスト 8.5 作成したスタイルファイルの使い方

```
1 import matplotlib.pyplot as plt
2
3 # スクリプトの冒頭で自分のスタイルシートを読み込む
4 plt.style.use("./my_style.mplstyle")
5
6 # 以降の描画はすべて統一されたスタイル (Serif フォント等) が反映される
7 fig, ax = plt.subplots(figsize=(6, 4))
```

このように設定をファイルに分離しておくことで、新しい解析スクリプトを書くたびにフォントやグリッドの有無がバラバラになる事態を防ぐことができる。また、学会発表用の大きなフォント設定といった別のファイルを容易に切り替えることも可能になる。

8.7 共通スタイルの一部だけを上書きする

共通スタイルを決めておくと便利だが、発表スライド用に文字だけ大きくしたい、下書きではグリッドだけ消したい、といった局所的な変更もよく起こる。そのたびに元の `my_style.mplstyle` を書き換えると、別の図まで見た目が変わってしまう。こういう場合は、基準となるスタイルを読み込んだ後に、必要な項目だけを重ねる方が管理しやすい。

```

# presentation_style.mplstyle
font.size: 16
axes.grid: False
legend.fontsize: 11

1 import matplotlib as mpl
2 import matplotlib.pyplot as plt
3
4 plt.style.use(["./my_style.mplstyle", "./presentation_style.mplstyle"])
5
6 # スタイルファイルを増やしたくないなら、その場で個別に上書きしてもよい
7 mpl.rcParams["axes.titlesize"] = 14
8
9 fig, ax = plt.subplots(figsize=(6, 4))
10 ax.grid(False) # この Axes だけグリッドを消す

```

二つの `.mplstyle` を配列で渡すと、後ろに書いたファイルの設定が優先される。したがって、基準となる `my_style.mplstyle` を保ったまま、文字サイズやグリッドの有無だけを差し替えられる。さらに、`mpl.rcParams` を直接書き換えれば、そのスクリプト以降の図だけを変更でき、`ax.grid(False)` のような Axes メソッドを使えば、単一のパネルにだけ適用できる。設定の効く範囲が、スタイルファイル、スクリプト全体、個別の Axes の三段階に分かれると理解しておくで混乱しにくい。

8.8 Matplotlib の数式表記を使う

Matplotlib では、軸ラベルや凡例に数式風の表記を書ける。既定では LaTeX そのものではなく、TeX に似た `MathText` という仕組みが使われる。

リスト 8.6 数式を含むラベル

```

1 plt.xlabel(r"$\Delta t$, [\mathrm{ns}]$")
2 plt.ylabel(r"$N$")

```

単位まで数式の中へ入れるなら、上の例のように `mathrm` を使って立体で書く方が自然である。レポートを LaTeX で書くなら、図中の記号も本文と整合するようにそろえると見通しがよい。

また、図中で使う記号は本文での定義と一致させるべきである。本文で τ を時定数として定義したなら、図の凡例やキャプションでも同じ記号を使う方が親切である。解析内容と図の表現が食い違っていると、読者は不要なところで迷う。

8.9 凡例、余白、複数パネル

複数の系列やフィット曲線を同じ図へ重ねるなら、凡例は不可欠である。凡例をどこへ置くか、枠を付けるか、何をどの順に書くかで、読みやすさは大きく変わる。

ここでいう `values1` と `values2` は、同じ物理量を別条件で測った 1 次元配列だと考えればよい。例えば、温度条件 A と B で得た信号振幅を左右のパネルへ並べ、同じビン数でヒストグラムを描いて分布形状を比較する場面である。単にコードの書式を見るのではなく、同じ量を別条件で見比べるための雛形だと捉えると、このリストの意図が分かりやすい。

リスト 8.7 凡例とレイアウトの例

```
1 fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
2 axes[0].hist(values1, bins=40, alpha=0.7, label="sample_A")
3 axes[0].set_title("sample_A")
4 axes[0].set_xlabel("value")
5 axes[0].set_ylabel("count")
6
7 axes[1].hist(values2, bins=40, alpha=0.7, label="sample_B")
8 axes[1].set_title("sample_B")
9 axes[1].set_xlabel("value")
10
11 for ax in axes:
12     ax.legend(loc="upper_right")
13     ax.set_axisbelow(True)
14
15 fig.tight_layout()
16 fig.savefig("comparison.pdf")
```

この例では、`axes` は二つの `Axes` を並べた配列であり、`for ax in axes:` の部分で「凡例位置をそろえる」という共通処理を両パネルへまとめて適用している。`sharey=True` を付けると縦軸の尺度がそろうため、左右の高さをそのまま比較しやすい。ここでは各パネルに系列が一つしかないため、凡例はやや大げさに見えるかもしれないが、後からフィット曲線や参照線を重ねるとそのまま拡張できる書き方である。

`tight_layout` は余白調整の基本であるが、すべてを自動で完璧に直してくれるわけではない。タイトルや長いラベルが多い場合は、図のサイズや余白を自分で調整した方がよい。複数パネルの図では、どのパネルを比較させたいのかを意識して配置することが大切である。

凡例がデータを隠すなら、`loc="best"` で自動配置に任せるか、`bbox_to_anchor` を使って枠の外へ逃がす。レイアウト調整は「自動で整うかどうか」ではなく、「比較したい情報が隠れていないかどうか」で判断する方が実践的である。

8.10 結果例の読み方

図を読めるようになるには、単に描けるだけでは足りない。まず軸と単位を確認し、次に分布の中心と幅、最後に裾や外れ値を見る、という順で眺めると整理しやすい。

例えば、図 8.1 の左パネルでは、ヒストグラムの山がフィット曲線とよく重なっているか、左右対称性がどの程度あるかを見る。右パネルでは、短時間側の立ち上がりや長時間側の減衰が同じ時定数で説明できるかを見る。こうした読み方を本文中で言葉にしておくと、図は単なる飾りではなく考察の根拠になる。

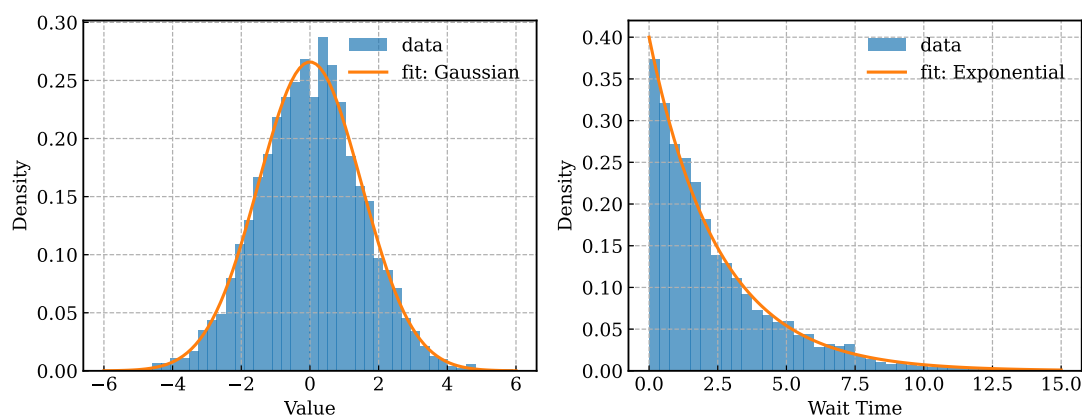


図 8.1 Matplotlib による分布図とフィットの例。左はガウシアン型、右は指数分布であり、どちらもヒストグラムの上にフィット曲線を重ね、凡例に関数形とパラメータを書いている。

8.11 画像フォーマットの選択：ラスタとベクタ

解析図を保存する際、適切な画像フォーマットを選ぶことは極めて重要である。

- **ベクタ形式 (.pdf, .svg, .eps)**：どれだけ拡大しても線や文字がぼやけない。論文やレポートで図を用いる場合、文字やプロット点を高品質なまま印刷できるためこれが基本となる。関数 `savefig("plot.pdf")` で保存できる。
- **ラスタ形式 (.png, .jpg)**：ピクセル（点）の集まりで画像を描画する。拡大するとジャギ（ピクセルのギザギザ）が見える。ただし、例えば数十万点に及ぶ密集した散布図や、ヒートマップのような巨大な 2 次元データをベクタ形式で保存すると、点の一つ一つがファイルに記録されるためファイルサイズがギガバイトを超え、PDF ビューアがフリーズする原因になる。そのような複雑な図の場合は、`savefig("plot.png", dpi=150)` のように解像度 (dpi) を指定して PNG で出力する方が適切である。

8.12 相関の可視化：論理的改善ステップ

データ解析において、2 つの変数の相関関係を見ることは非常に多い。しかし、これを「とりあえず」描画関数にかけるだけでは、重要な特徴を見落とす。データ量に応じた適切な図の選択プロセスを順番に見ていく。

8.12.1 Step 1: 直感的な散布図 (Scatter) の限界

2 変数の相関を見るのに最も基本となるのが散布図 (scatter) である。しかし、データ点が数十万規模になると、これはすぐに「役に立たない図」と化す。

リスト 8.8 大量データにおける散布図

```
1 # s=10 で大量の点（約60万点）をそのまま描画すると...
```

```

2 fig, ax = plt.subplots(figsize=(6, 5))
3 ax.scatter(x, y, s=10, c='black')
4 fig.savefig("scatter_bad.png", dpi=150)

```

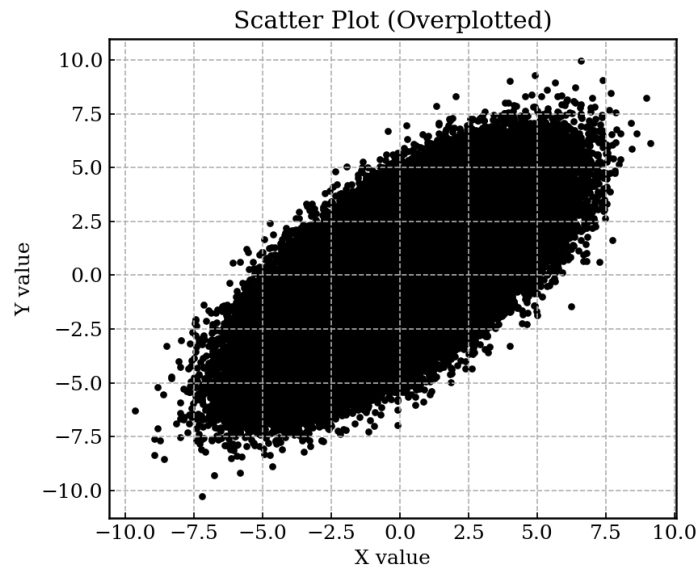


図 8.2 大量データに対する散布図。全てが黒く沈んでおり内部構造が一切見えない。

図 8.2 のような図を作ってしまったてはならない。データがどこにどれだけ集中しているのか（密度）が全く分からないからである。

8.12.2 Step 2: 2次元ヒストグラムの導入と、線形スケールの限界

点が潰れるなら、「その領域にいくつ点があるか」を色で表現する 2 次元ヒストグラム (hist2d) が有効である。

リスト 8.9 線形カラーバーを用いた 2 次元ヒストグラム

```

1 fig, ax = plt.subplots(figsize=(6, 5))
2 h_lin = ax.hist2d(x, y, bins=50, cmap='viridis')
3 cbar = fig.colorbar(h_lin[3], ax=ax, label="Counts (Linear)")
4 fig.savefig("hist2d_linear_bad.pdf")

```

図 8.3 は中心部が明るくなっているため、散布図よりはマシに見える。しかし、極端に集中している一部（例: 2 万カウント）の色に引っ張られ、10 カウント、100 カウントといった周囲のノイズや裾野の広がりがすべて濃紺に塗りつぶされてしまっている。

8.12.3 Step 3: 大規模データに必須となる LogNorm (対数カラスケール)

現実の物理イベントは、頻度が「1, 10, 100, 1000」のように桁単位で変化することが多い。この何桁にもまたがる広がりをもつ一つの図に収めるための判断基準が「カウントのカラーバーを対数 (log) にするべきか否か」である。

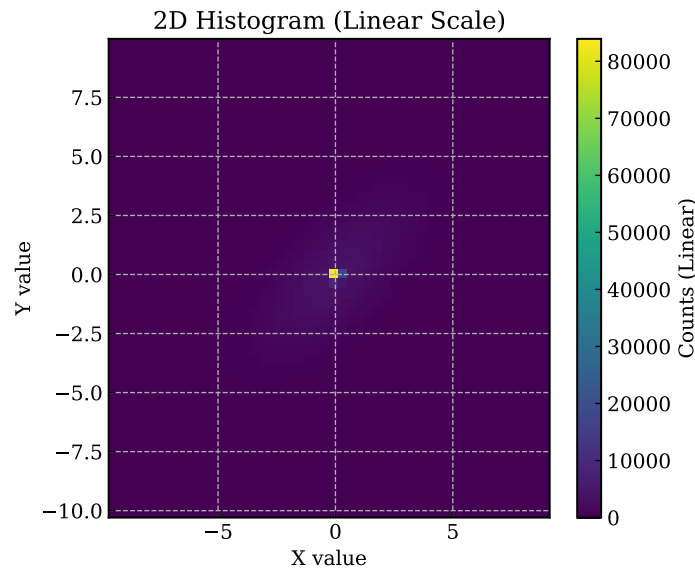


図 8.3 線形スケールを用いた 2D ヒストグラム。最も集中した中心部だけが目立ち、周辺が潰れる。

リスト 8.10 LogNorm を用いた適切な 2 次元ヒストグラム

```
1 import matplotlib.colors as mcolors
2
3 fig, ax = plt.subplots(figsize=(6, 5))
4 # norm=mcolors.LogNorm() により色の割当を対数スケールにする
5 h_log = ax.hist2d(x, y, bins=50, norm=mcolors.LogNorm(), cmap='viridis')
6 cbar = fig.colorbar(h_log[3], ax=ax, label="Counts (Log Scale)")
7 fig.savefig("hist2d_lognorm_good.pdf")
```

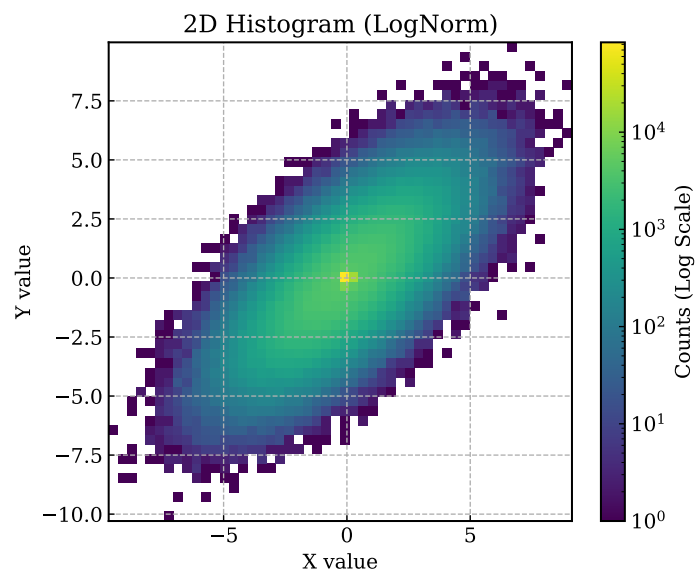


図 8.4 LogNorm を用いた 2D ヒストグラム。中心の密集部分と、周囲の広がりが同時に視認できる。

図 8.4 を見れば、単一のピークの周りに広く緩やかなイベントの相関層が広がっていることが一目で分かる。これが「なぜ LogNorm を知っていなければならないか」の強力な理由

である。

8.13 複数の図を並べる (subplots)

レポートや論文では、関連する分布を左右や上下に並べて比較することが多い。この場合、1枚のキャンバス (Figure) の中に複数のパネル (Axes) を配置する機能を利用する。

リスト 8.11 複数のパネルを並べる方法

```
1 # 1行2列（左右に2つ）のパネルを作成する。
2 # fig はキャンバス全体、axes は2つのパネル（配列）を表す。
3 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
4
5 # 左のパネルには axes[0] を使って描画する
6 axes[0].hist(data_a, bins=30)
7 axes[0].set_title("Data_A")
8
9 # 右のパネルには axes[1] を使って描画する
10 axes[1].hist(data_b, bins=30)
11 axes[1].set_title("Data_B")
12
13 # 複数のパネルを描くと軸の数字やラベルが隣の図と重なりやすいため、
14 # 最後に必ず tight_layout() を呼んで余白を自動調整する。
15 fig.tight_layout()
16 fig.savefig("comparison.pdf")
```

単純な `plt.plot()` ばかり使っていると、この「特定のパネル (`axes[0]`) に対して描画を命令する」という書き方に慣れるのに少し時間がかかるが、複雑な比較図を作る上では避けて通れない技法である。

8.14 エラーバーと対数軸 (log-log プロット)

実際の物理データの多くは測定誤差を伴うため、単なる散布図ではなくエラーバー付きのプロットが必要になる。また、横軸や縦軸のスケールが指数関数的に落ち幅を変える場合、対数 (log) のグラフでプロットしなければ分布の様子が潰れてしまうことが多い。

図 8.5 にエラーバーと対数プロットの例を示す。

リスト 8.12 エラーバーと対数軸プロットの詳細解説

```
1 fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 横10インチ、縦4インチで 1行2列の
  パネルを作成
2
3 # 左のパネル (axes[0]) へのプロット
4 # x_val, y_val を点ごとにエラー y_err でプロット。
5 # fmt='o' は丸い点を意味し、capsize=3 はエラーバーの終端の横棒の長さ。
6 axes[0].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
7 axes[0].set_xlabel("Time[ns]")
```

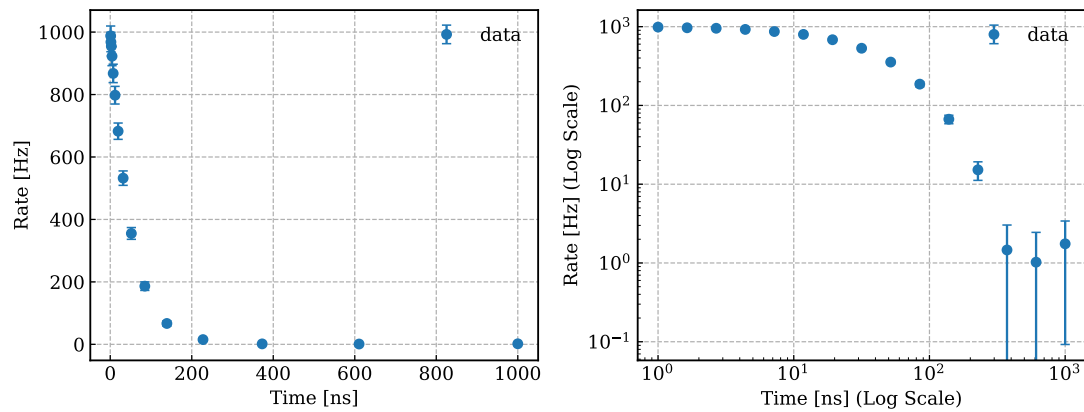



図 8.5 エラーバーと対数軸の設定例。左は通常の線形軸、右は X, Y 両方を対数にした図。

```

8
9 # 右のパネル (axes[1]) へのプロット
10 axes[1].errorbar(x_val, y_val, yerr=y_err, fmt='o', capsize=3, label="data")
11 # 対数スケール化
12 axes[1].set_yscale('log')
13 axes[1].set_xscale('log')
14
15 fig.tight_layout() # ラベルや目盛りが重ならないように余白を自動調整
16 fig.savefig("errorbar_example.pdf")

```


第 9 章 SciPy とフィット

9.1 フィットとは何か

ヒストグラムを描いただけでは、分布の特徴を定量化できないことがある。そこで、ある関数形を仮定してパラメータを求める。これがフィットである。

フィットは万能ではない。まず、なぜその関数形を選ぶのかという物理的、統計的な理由が必要である。さらに、ヒストグラムの範囲、ビン幅、外れ値処理によって見え方が変わるため、「フィット結果が出た」ことと「解釈してよい」ことは別である。

フィットの目的は、曲線をきれいに重ねることではない。分布の中心、幅、時定数などを、比較可能な量として取り出すことが目的である。したがって、フィット前に図を見ておくこと、フィット後に残差やずれを確認することが重要になる。

9.2 この本でよく使う分布

データ解析では、ガウシアンと指数分布がよく現れる。

- ガウシアン: 時刻差のばらつき
- 指数分布: 待ち時間や発生間隔

もちろん、実際のデータがいつもきれいな理想分布になるとは限らない。重要なのは「なぜその関数を当てようとしたのか」を説明することである。

分布の名前を覚えるだけでなく、どの量に対して自然に現れるかを知ることが重要である。ガウシアンは独立なばらつきの和として、指数分布はランダムな待ち時間として現れやすい。こうした背景を知っていると、単なる形合わせで終わらない。

9.3 SciPy の使い方

SciPy には、分布やフィットのための道具が複数ある。代表的なのは `scipy.stats` と `scipy.optimize.curve_fit` である。図 8.1 のような曲線を重ねると、単に形を見るだけでなく、平均や幅、時定数を数値として比較しやすくなる。

単純な場合は、自分で平均や分散を計算するだけでも十分なことがある。一方で、より一般的なモデルや誤差評価をしたいなら SciPy が役立つ。

`scipy.optimize.curve_fit` を使うと、モデル関数を自分で定義してパラメータを推定で

きる。最初は少し抽象的に見えるが、関数形を明示してデータへ当てるという流れを一度理解すると、応用範囲は広い。

リスト 9.1 ガウシアン の平均と幅を求める最小例

```
1 import numpy as np
2
3 values = np.array([1.2, 0.9, 1.5, 1.1, 0.8])
4 mean = values.mean()
5 sigma = values.std(ddof=0)
6 print(mean, sigma)
```

リスト 9.2 curve_fit の最小例

```
1 from scipy.optimize import curve_fit
2
3 def linear_model(x, a, b):
4     return a * x + b
5
6 params, cov = curve_fit(linear_model, xdata, ydata)
```

params には推定パラメータが入り、cov には共分散行列が入る。後者まで見る習慣を付けておくと、推定値の不確かさをどう扱うかという次の段階につながる。

リスト 9.3 フィット結果の出力例

```
mean = 3.02
sigma = 0.79
tau = 2.47
```

こうした数値が得られたら、次に考えるべきは「図の見た目と整合するか」「別条件のデータと比較して意味があるか」「単位は妥当か」である。数値が出たこと自体より、どう読むかの方が重要である。

SciPy を使うときも、ライブラリ名だけを覚えるのではなく、「どの関数が何を入力として何を返すか」を確認する読み方が重要である。これは Python 公式チュートリアル の標準ライブラリ章を読むときと同じ姿勢である。

9.4 Astropy で時刻を扱う

宇宙線や天体観測の解析では、単なる文字列や datetime だけでは扱いにくい時刻表現がよく現れる。例えば UTC を前提にした観測時刻、MJD (Modified Julian Date)、Unix time の相互変換である。こうした場面では astropy.time.Time が便利である。

リスト 9.4 Astropy の Time を使う例

```
1 from astropy.time import Time
2
3 t_event = Time("2026-04-04_12:34:56.123", format="iso", scale="utc")
4 print(t_event.mjd)
5 print(t_event.unix)
```

`datetime` が不足しているわけではないが、観測データの時刻系をまたいで扱うなら Astropy の方が自然なことが多い。特に、MJD や JD を使うデータを手で換算すると誤差や解釈のずれが入りやすい。時刻表現の変換は、自前で式を書かずに専用ライブラリへ任せた方が安全である。

さらに、`astropy.coordinates` を使うと、赤道座標、銀河座標、地平座標などの変換も行える。座標変換そのものを本書で詳しく扱うわけではないが、宇宙物理の解析では Astropy がその入口になることを知っておく価値がある。

9.5 Astropy で単位を扱う

物理量を扱うときは、数値だけでなく単位も一緒に管理した方が安全である。エネルギー、時間、距離を別々の単位系で持っている、換算漏れがそのままバグになる。Astropy の `units` を使うと、値に単位を持たせたまま計算できる。

リスト 9.5 Astropy の `units` を使う例

```
1 import astropy.units as u
2
3 speed = 100 * u.km / u.s
4 time = 5 * u.min
5 distance = speed * time
6 print(distance.to(u.km))
```

単位付き計算は一見まわりくどく見えるが、研究ノートの段階ではむしろ安心である。最終的に高速化や大量処理のために単位を外すことはあっても、まず式の意味を確認する段階では、単位を持たせておく方が解釈ミスを減らせる。

9.6 フィットの実務的な流れ

実際の解析では、次の順に進めると混乱しにくい。

1. まず図を描き、どの関数形が候補かを考える
2. フィット範囲を決める
3. 初期値が必要なら妥当な値を与える
4. フィット曲線を図へ重ねる
5. 推定値と残差を確認する

この順序を踏まずにいきなり `curve_fit` を呼ぶと、得られた数値が妥当なのか判断しにくい。フィットは関数呼び出しではなく、仮説と検証の手続きとして理解した方がよい。

9.7 初期値と制約

モデルによっては、初期値の与え方で収束先が変わることがある。また、明らかに負になってほしくないパラメータなどには制約を入れた方が自然なこともある。

リスト 9.6 初期値と bounds の例

```
1 params, cov = curve_fit(  
2     model,  
3     xdata,  
4     ydata,  
5     p0=[1.0, 0.5],  
6     bounds=([0.0, 0.0], [np.inf, np.inf]),  
7 )
```

p0 は初期値、bounds はパラメータ範囲である。どちらも乱用すべきではないが、物理的にあり得る範囲が分かっているなら、フィットの安定化に役立つ。

フィット結果を図へ重ねるときは、関数形と推定パラメータを凡例に書く方が見やすいことが多い。図の上に直接テキストを置くと、データや曲線と重なって読みにくくなりやすい。特に、比較する図が複数ある場合は、凡例の書式をそろえると見通しが良くなる。

リスト 9.7 フィット結果を凡例へ書く例

```
1 fig, ax = plt.subplots()  
2 ax.hist(values, bins=40, density=True, alpha=0.7, label="sample")  
3 ax.plot(  
4     x,  
5     gaussian_pdf(x, mean, sigma),  
6     linewidth=2.0,  
7     label=(  
8         "fit:  $\frac{1}{\sqrt{2\pi}\sigma}$   
9          $\exp\{-\frac{(x-\mu)^2}{2\sigma^2}\}$   
10        "  
11        "\n"  
12        + fr"$\mu={mean:.2f}, \sigma={sigma:.2f}$"  
13    ),  
14 )  
15 ax.legend(loc="upper right")
```

図 8.1 でも、フィット曲線の式と推定パラメータは凡例へ入れている。本文、キャプション、凡例の役割を分けると、図全体が読みやすくなる。

9.8 残差とフィット範囲

フィットが見た目に良さそうでも、特定の範囲だけ大きくずれていることがある。そのため、可能なら残差も確認したい。

リスト 9.8 残差を調べる考え方

```
1 model_y = gaussian_pdf(x, mean, sigma)
2 residual = y - model_y
```

また、外れ値や非物理的な領域を含めると、パラメータが不安定になることがある。どの範囲でフィットしたかを書き残すことは、再現性のためにも解釈のためにも重要である。

リスト 9.9 フィット範囲を制限する例

```
1 mask = (xdata >= 0.0) & (xdata <= 10.0)
2 params, cov = curve_fit(model, xdata[mask], ydata[mask])
```

フィット範囲を切るという行為は、都合の悪い部分を隠すことではなく、どの領域でそのモデルが有効だと考えるかを明示することである。したがって、結果を報告するときは範囲も本文に書くべきである。

9.9 フィット時の σ (重み) の重要性

物理の実データでは、各データポイントが常に同じ信頼度を持つとは限らない。カウント数が少ない点は誤差（ポワソン誤差）が大きく、カウント数が多い点は誤差が小さい。この情報を無視してフィットを行うと、信頼性の全くない外れ値に引っ張られて誤った結果を生む。

9.9.1 Step 1: 誤差を無視した素朴なフィットの限界

まずは、各データ点を持つエラーバー（誤差）を全て同じとみなしてフィットしてみよう。

リスト 9.10 誤差の大きさを考慮しないフィット

```
1 from scipy.optimize import curve_fit
2
3 def linear_model(x, a, b):
4     return a * x + b
5
6 # sigma を指定せず、単に点と点の距離だけで合わせようとする
7 popt, bad, cov = curve_fit(linear_model, xdata, ydata)
```

9.9.2 Step 2: σ による適切な重み付け

正しいアプローチは、「誤差が大きい（不確かな）点には合わせに行かず、誤差が小さい（確かな）点に重みをおいて」フィットすることである。これを `curve_fit` における `sigma` 引数で指定する。

リスト 9.11 σ を用いた適切な重み付きフィット

```
1 # sigma=y_err で各点の重みを伝える。
2 # absolute_sigma=True にすることで、誤差が物理的な絶対値であることを教える（共分散の計算に影響）
```

```

3 popt_good, cov = curve_fit(
4     linear_model,
5     xdata,
6     ydata,
7     sigma=y_err,
8     absolute_sigma=True
9 )
10
11 # パラメータの誤差は共分散行列の対角成分の平方根から算出される
12 errors = np.sqrt(np.diag(cov))

```

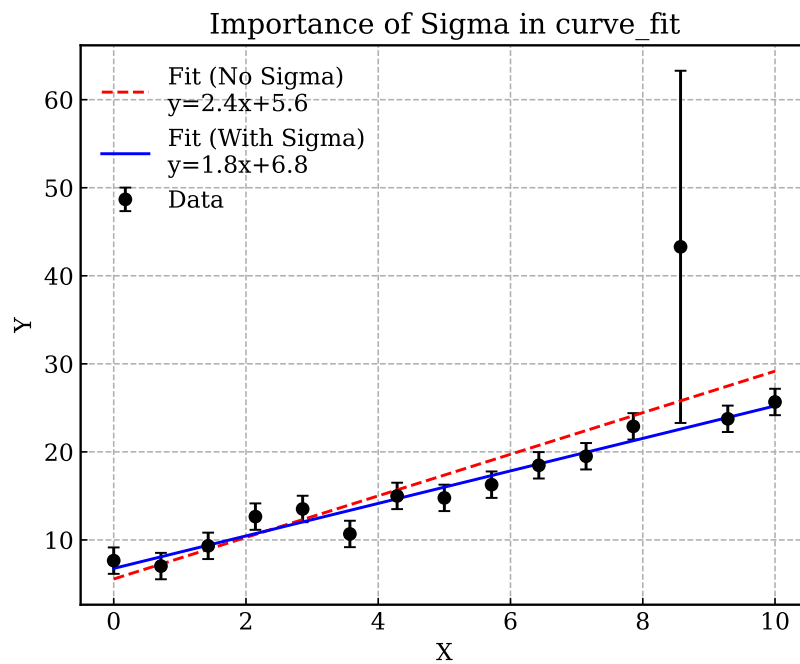


図 9.1 sigma の有無によるフィット結果の激変。中央の点に極端に大きなエラーバー（不確かな値）がある場合、sigma を無視したフィット（赤点線）はこれに引っ張られて真の傾きから大きく外れるが、sigma を指定したフィット（青実線）は不確かな点を無視して正しい傾向を掴んでいる。

図 9.1 から明らかなように、‘sigma’ の指定を怠ると結果全体が台無しになる。

第 10 章 Python 解析の高速化

10.1 まず測る

高速化を始める前に、どこが遅いのかを測る。読み込みが遅いのか、探索が遅いのか、図の作成が遅いのかを区別しないまま最適化しても、効果は見えにくい。

ここで役立つのが `time.perf_counter` である。処理の前後で時刻を取り、差を記録するだけでも、どの段階が重いのかの見当が付く。大切なのは、全体時間だけでなく、読み込み、前処理、探索、書き出し、作図のように段階を分けて測ることである。

リスト 10.1 処理時間を測る最小例

```
1 import time
2
3 t0 = time.perf_counter()
4 run_analysis()
5 t1 = time.perf_counter()
6 print(f"elapsed={t1-t0:.3f}s")
```

測定するときは、一度だけの結果を絶対視しないことも重要である。キャッシュや OS の状態で多少は揺れるため、条件をそろえ、複数回の傾向を見る方がよい。

10.2 Jupyter では %timeit も使える

ノートブック環境で短い式や関数の速度を見たいときは、Jupyter の `%timeit` や `%%timeit` も便利である。複数回の実行を自動で繰り返し、平均的な実行時間を表示してくれるため、小さな差を見たいときに役立つ。

リスト 10.2 Jupyter の timeit の例

```
1 %%timeit
2 result = array * 2
```

ただし、`%timeit` はノートブックや IPython での簡便な測定であり、スクリプト全体の読み込みや入出力を含めた時間を見るには向かない。小さな部品には `%timeit`、解析全体には `perf_counter` というように使い分けると整理しやすい。

10.3 計算量の見積もり

高速化では、実測だけでなく理屈の見積もりも役に立つ。入力サイズを N としたとき、処理回数が N に比例するのか、 $N \log N$ 程度なのか、 N^2 なのかで、大きなデータに対する伸び方がまるで違う。

厳密な漸近解析までできなくても、「データを 2 倍にしたら計算量は何倍くらいになるか」を考えるだけで十分に有益である。高速化はテクニック集ではなく、問題の構造を考える作業である。

リスト 10.3 計算量の見方の例

$O(N)$: 1 回走査 $O(N \log N)$: ソートを含む処理 $O(N^2)$: 全組合せ比較

この違いは、小さなテストデータでは目立たなくても、大きな入力で急に効いてくる。だからこそ、実測の前に理屈として見積もる意味がある。

10.4 遅い例と速い例を比べる

同じ問題でも、書き方の差ではなく、アルゴリズムの差で桁違いに時間が変わることがある。この章では、その違いを具体例で見る。図 10.1 は、単純な隣接差分フィルタを純粋な Python ループ、NumPy、pandas で処理した例である。

この点は初学者ほど見落としやすい。変数名を短くしたり、関数を減らしたりすることは、ほとんど速度に効かない。一方で、比較回数を減らす、探索範囲を絞る、配列演算へ置き換えるといった変更は、本質的に計算量を変えることがある。

10.5 ペア探索の遅い例

時刻が昇順に並んだ値が N 個あるとき、すべての組 (i, j) を比べる単純な二重ループは、おおよそ N^2 に比例する計算になる。

リスト 10.4 遅いペア探索の例

```
1 for i in range(len(times)):
2     for j in range(i + 1, len(times)):
3         if abs(times[j] - times[i]) <= window:
4             ...
```

この書き方は分かりやすいが、データ量が増えたとすぐに実用でなくなる。

二重ループが悪いのではなく、「不要な比較まで全部やっている」ことが問題である。小さな入力では問題なくても、データ量が 10 倍になると比較回数はおおよそ 100 倍になる。この増え方を直感として持っておくと、高速化の議論が理解しやすい。

10.6 ペア探索の速い例

時刻順に並んでいる列なら、遠すぎる要素まで比べる必要はない。差が閾値を超えた時点で比較を打ち切る、あるいは隣接要素だけを見る、といった工夫で計算回数を大きく減らせる。

実際には、どこまで比較すれば十分かは問題設定に依存する。だが共通しているのは、「入力の構造を利用する」という発想である。ソート済み、単調増加、局所性がある、といった性質が分かっているなら、それを使わないのはもったいない。

10.7 トリプレット探索を題材にしたアルゴリズムの差

トリプレット探索では、単純な三重ループはさらに厳しい。ここで重要なのは、組合せを力づくで全部試すのではなく、近傍候補を先に絞り、そこから有効な組だけを組み立てることである。

この視点は、ペア探索にも一般のクラスタ探索にも共通している。

アルゴリズムを考えると、まず最も素朴な実装を書いて正しさを確かめ、その後で不要な組合せを減らす方法を探すとよい。最初から複雑な高速版だけを書こうとすると、速いが間違っているコードになりやすい。

10.8 NumPy と pandas による高速化

アルゴリズムを見直した後は、NumPy や pandas へ処理を移すことで、Python レベルのループを減らせる。特に差分計算、ソート、列抽出、フィルタは、これらのライブラリが得意とする。図 10.1 から、配列演算が速度に与える効果を視覚的に確認できる。

ここで注意したいのは、NumPy や pandas を使えば自動的に何でも速くなるわけではないことである。根本的に無駄な比較をしているなら、配列演算へ移しても限界がある。まず計算量を減らし、そのうえで実装をベクトル化するという順序が自然である。

10.9 メモリとコピーの管理

速度だけを見ていると、メモリ使用量が大きくなりすぎて逆に遅くなることがある。巨大な中間配列を何度も作る実装では、CPU よりもメモリ転送が支配的になる場合がある。

そのため、高速化では「何回ループしたか」だけでなく、「何回コピーしたか」も見る必要がある。不要なコピーを減らし、必要な列や範囲だけを扱う設計は、速度と安定性の両方に効く。

10.10 分割読み込み、並列化、ストリーミング

大規模解析では、全部を一度にメモリへ載せない設計も重要である。ファイル単位で処理し、必要な情報だけを持ち回る。さらに、独立な単位に分割できるなら並列化も検討できる。

ストリーミング処理では、「今必要な範囲だけを覚える」という考え方が中心になる。全履歴が不要なら保持しない、集計済みの量は逐次更新する、といった設計が有効である。メモリ節約と速度向上が同時に得られることも多い。

10.11 プロファイラを使う

本格的に遅い理由を調べるなら、プロファイラも役立つ。例えば `cProfile` を使うと、どの関数に時間が使われているかを確認できる。

リスト 10.5 `cProfile` の最小例

```
1 import cProfile
2
3 cProfile.run("run_analysis()", sort="cumtime")
```

ただし、プロファイラの数字も文脈なしには読めない。回数が多いから遅いのか、1 回が重いのか、入出力を待っているのかを区別する必要がある。測定値とコード構造を一緒に見ることが重要である。

リスト 10.6 プロファイル結果の見方の例

```
ncalls tottime cumtime function
100000 1.25 1.40 check_pair
10 0.02 2.10 read_all_files
```

この結果例では、`check_pair` は 1 回は軽いが回数が多く、`read_all_files` は内部の処理込みで累積時間が大きいと読める。どの列が何を意味するかを理解すると、改善方針が立てやすくなる。

10.12 C や C++ へ書き換えるという選択肢

Python は開発速度が高いが、計算コアのすべてを Python で書く必要はない。アルゴリズムを十分に整理したうえで、どうしても重い部分だけを C や C++ へ移す、というのは自然な選択肢である。

ただし、言語を変える前に Python 側で何が遅いかを把握しておく必要がある。ボトルネックが入出力や高水準の設計にあるなら、C++ 化しても大きな改善にならない。低レベル言語への移行は、最後の切り札として考える方がよい。

リスト 10.7 重いループを C++ へ出す発想の例

```

1 for (std::size_t i = 0; i < n; ++i) {
2     for (std::size_t j = i + 1; j < n; ++j) {
3         if (std::abs(times[j] - times[i]) <= window) {
4             ++count;
5         }
6     }
7 }

```

この書き換えは、定数倍の改善をもたらすことがある。しかし、比較回数そのものが多すぎるなら本質的な改善ではない。アルゴリズムと実装言語を分けて考える姿勢が必要である。

10.13 Python から C/C++ を呼び出す

方法はいくつかある。たとえば ctypes で共有ライブラリを呼ぶ方法、Python C 拡張を書く方法、pybind11 を使って C++ を Python から自然に使える形へ包む方法などがある。さらに、Python と C の中間に位置する Cython を使い、Python に近い構文のまま重いループだけをコンパイルする方法も、この節に含まれる重要な選択肢である。

リスト 10.8 Cython で型を付ける発想の例

```

1 cdef int i
2 cdef double total = 0.0
3
4 for i in range(n):
5     total += values[i]

```

どの方法を選ぶかは、必要な速度、保守性、ビルドの難しさで決まる。短い試作なら ctypes、Python らしい使い勝手まで整えたいなら pybind11、既存の Python 風コードを保ちながら内側だけ速くしたいなら Cython、といった選択が考えられる。重要なのは、インタフェースを狭く保ち、重い部分だけを外へ出すことである。

リスト 10.9 Python 側の呼び出しイメージ

```

1 result = fast_module.count_pairs(times, window)

```

Python 側の呼び出しが単純であれば、内部実装を後から差し替えやすい。まず Python で正しい API を決め、その内部だけを高速化するという順が自然である。なお、Cython も強力だが、アルゴリズムそのものが無駄な場合には決定打にならない。この点は C++ 化と同じであり、まず Python 側で何が遅いのか、どのループが本当に支配的なのかを見極めた後に使う方が効果的である。

10.14 外部実装を参考にする時の見方

高速化を学ぶときは、外部の C や C++ の実装例を読むことが参考になる。ただし、そこで大切なのは固有名詞を覚えることではない。まず見るべきなのは、どの問題を解いている

のか、どのアルゴリズムを選んでいるのか、どの部分だけを低レベル言語へ移しているのか、という点である。

高速化の章では、このような例を参照しながら、次の二つを常に区別して考える。

- アルゴリズムを変えたから速くなったのか
- 実装言語を変えたから速くなったのか

多くの場合、最初に効くのは前者である。

したがって、高速化のレポートを書くときは、「NumPy にした」「C++ にした」で終わらせない方がよい。何が変わった結果として速くなったのかを、計算量、比較回数、メモリ配置、ループの位置といった観点で説明するべきである。

10.15 結果例と報告の仕方

高速化の結果は、本文中で比較できる形に整理すると読みやすい。例えば、次のような簡潔な結果でも十分に意味がある。

リスト 10.10 速度比較の結果例

```
slow version : 182.4 s
vectorized   : 12.7 s
streaming    : 4.8 s
```

この表現だけでは理由が分からないので、続けて「比較回数を減らした」「Python ループを配列演算へ移した」「全件保持をやめた」といった説明を添える。数値と理由の両方がそろって初めて報告になる。

リスト 10.11 隣接差分を NumPy でまとめて計算する例

```
1 import numpy as np
2
3 values = np.array(data, dtype=np.float64)
4 diffs = np.diff(values)
5 count = np.count_nonzero(diffs > 0.25)
```

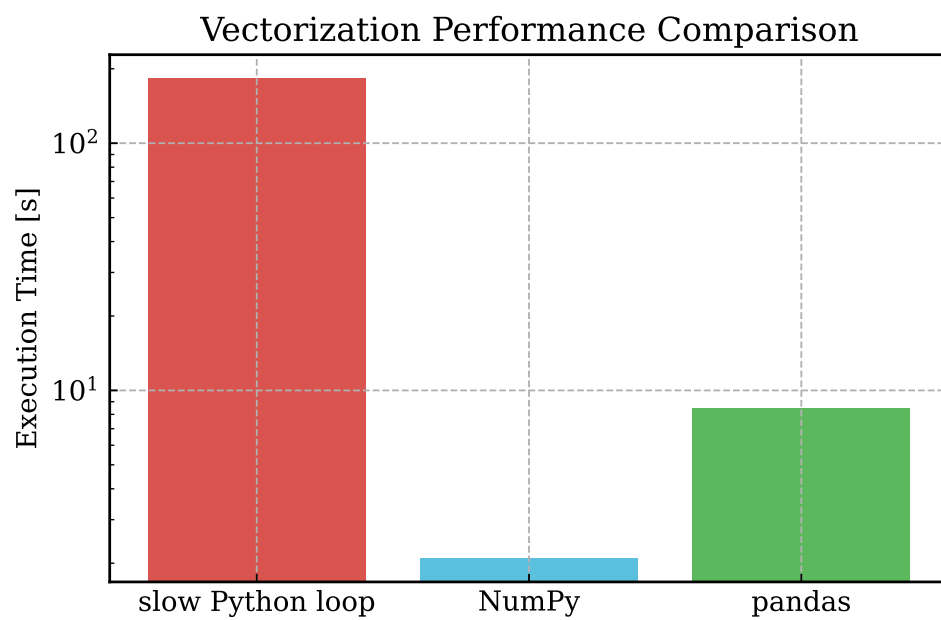


図 10.1 同じ隣接差分フィルタを、純粋な Python ループ、NumPy、pandas で実行した例。縦軸は実行時間で、対数目盛を使っている。

第 11 章 課題: 同時観測イベントの探索

11.1 配布ファイル

この課題では、次のファイルを配布する。

- events_data.zip
- slow_event_analysis.py
- 本テキスト

events_data.zip を展開すると、events_data/ というディレクトリが得られる。日ごとに gzip 圧縮されたイベントファイルがあり、年・月ごとのディレクトリに分かれている。

課題のスクリプトやデータは、Google Drive (https://drive.google.com/drive/folders/19RWzbiXUkj5Dgbv0wiu0I-bvEsW8nRmz?usp=drive_link) から入手できる。手元に配布物がそろっていない場合は、まずここから取得して配置を確認する。

配布物は意図的に最小限にしてある。したがって、何を入力とし、どのファイルが正解判定の対象で、どこからどこまでを自分で実装するのかを最初に整理してから着手した方がよい。

11.2 まず配布コードを動かす

まずは、配布された遅いコードをそのまま動かす。

リスト 11.1 配布コードの実行

```
$ python3 slow_event_analysis.py events_data --outdir slow_output
```

この実行は環境によってかなり時間がかかる。最初は数日分だけ切り出して試してもよい。

重要なのは、最初から完成版を書こうとしないことである。まず配布コードが本当に遅いことを自分の環境で確認し、どの出力が得られるかを見てから変更を始める方が、修正の意味が分かりやすい。

11.3 gzip ファイルの中身を見る

データの中身を見ずに解析を始めてはいけない。まずは 1 ファイルを開き、列の意味を確認する。

リスト 11.2 中身の確認

```
$ gzip -cd events_data/2024/01/events_2024_01_01.txt.gz | head
```

各行の列は次のようになっている。

```
event_id year month day hhmmss ns detector_id signal
```

列名を見たら、それぞれの型と役割を自分で言葉にしてみるとよい。整数として扱うべき列、時刻を構成する列、識別子としてだけ使う列、物理量として解釈する列が混ざっている。ここを曖昧にしたまま進むと、後でバグの原因になる。

この課題では、2 事象の時刻差を ns 単位まで含めて評価し、 $|\Delta t| \leq 250 \text{ ns}$ を満たすものを「同時観測」と定義する。したがって、hhmmss 列だけを見ればよいわけではなく、ns 列も含めて時刻を一つの量として組み立てる必要がある。秒境界や日付境界をまたぐ場合にどう振る舞うかも、この定義を満たす形で確認しなければならない。

11.4 小さいデータで確認する

いきなり 500 日分全体に対して試すのではなく、まずは数日分を別ディレクトリにコピーして試すとよい。

リスト 11.3 数日分だけを取り出す例

```
$ mkdir -p events_data_small/2024/01
$ cp events_data/2024/01/events_2024_01_01.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_02.txt.gz events_data_small/2024/01/
$ cp events_data/2024/01/events_2024_01_03.txt.gz events_data_small/2024/01/
```

小さい入力での確認は、速度のためではなく、正しさを素早く確かめるためである。数十秒で終わる条件を作っておけば、修正してすぐ再実行できる。大きなデータを回すのは、その後で十分である。

リスト 11.4 小さいデータでの確認結果の例

```
n_pairs_total = 241
analysis_sec = 18.4
```

この段階では、数値そのものよりも、修正のたびに一貫した結果が出るかが大切である。件数が毎回変わる、出力ファイルの形式が揺れる、といった状態では、その先の高速化に進むべきではない。

11.5 課題でやってほしいこと

最低限、次の内容に取り組むこと。

1. 配布コードの処理時間を確認する
2. 同時観測イベント数が正しいかを検証する

3. バグを修正する
4. 高速化する
5. 同時観測イベントの時刻差分布を描き、フィットする

この並びにも意味がある。いきなり高速化から始めるのではなく、まず元のコードの振る舞いを把握し、次に正しさを確認し、その後で改善する。間違ったコードを高速化しても、間違いが速くなるだけである。

11.6 出力としてほしいもの

最低限ほしい出力は次の三つである。

- 同時観測イベントの一覧
- 同時観測イベントを構成する 2 事象の時刻差の分布
- 同時観測イベントどうしの発生間隔、またはそれに相当する量の分布

ここでいう「2 事象の時刻差」は、同時観測と判定した 2 つのイベントの時刻差である。たとえば $\Delta t = t_2 - t_1$ と定義してよい。ただし、符号付きで使うのか絶対値を使うのか、検出器番号で向きをそろえるのかなどは、自分で決めて明記すること。

分布を描くときは、ビン幅、横軸の範囲、単位も明記すること。図は出しただけでは不十分で、どの量をどう定義して数えたのかが分からなければ、他人は解釈できない。フィットを行うなら、どの範囲を使ったかも書くべきである。

リスト 11.5 出力ファイル名の例

```
coincidence_pairs.csv
pair_dt_hist.pdf
pair_mean_gap_hist.pdf
summary.txt
```

ファイル名の流儀を最初に決めておくと、後で解析を繰り返したときにも混乱しにくい。どのファイルが一覧で、どれが図で、どれが数値要約なのかが名前から分かるようにしておくといよい。

11.7 正しさ確認の目安

今回の配布データは 500 日分であり、正しく同時観測イベントを拾えていれば、全期間で得られる同時観測イベント数は

```
n_pairs_total = 12033
```

になる。この値は最終確認の目安として使ってよい。

ただし、この値に合ったからといって、それだけで実装が正しいとは限らない。偶然件数だけ一致している可能性もある。可能なら、個々のイベント対の内容や、ファイル境界付近

の振る舞いも確認した方がよい。

リスト 11.6 summary.txt の例

```
n_pairs_total = 12033
analysis_sec = 248.6
plot_sec = 1.8
```

このような要約ファイルを出しておくと、コード変更のたびに結果を比較しやすい。件数だけでなく処理時間も同じ場所へ残すと、正しさと速度を一緒に追える。

11.8 提出物

提出物は次の二つである。

- 解析に使った Python コード
- LaTeX で書いたレポートと、その PDF

レポートには、少なくとも次の内容を書くこと。

- 元のコードの問題点
- 正しさをどう検証したか
- バグをどう修正したか
- どう高速化したか
- 改善前後の処理時間
- 作成した図
- フィット結果とその解釈

レポートを書くときは、変更点の羅列よりも、問題の発見、原因の切り分け、修正方針、検証方法、結果、考察の順で並べると読みやすい。図や表を入れるときは、本文の中で参照しながら説明すること。

リスト 11.7 レポート構成の例

1. 元のコードの観察
2. バグの特定
3. 修正内容
4. 高速化方針
5. 結果の比較
6. 図とフィットの考察

この順序で書けば、読者は「何が問題で、どう直して、何が改善したのか」を追やすい。レポートは成果物であると同時に、思考過程の記録でもある。

11.9 発展課題

余裕があれば、図の作成やフィットまで含めた全体処理をさらに高速化してよい。さらに、NumPy や pandas を使うだけでなく、自分でアルゴリズムを設計し直したり、C や C++ で計算コアを実装したりしてもよい。

この発展課題では、特定の解法を要求しない。pandas を深く使う方向もあれば、ストリーミング処理を突き詰める方向もあり得るし、重い部分だけを別言語へ切り出す方向もあり得る。重要なのは、自分の方針を説明できることである。

付録 A SSH, 鍵認証, ファイル転送

A.1 SSH とは何か

SSH は、離れたマシンへ安全に接続してコマンドを実行するための仕組みである。研究室や計算サーバーへ入り、大きなデータ処理や長時間ジョブを回すときによく使う。GitHub へ SSH で接続するときにも同じ仕組みを使うが、GitHub はシェルへログインする相手ではなく、Git 操作の認証先である点は区別しておいた方がよい。

A.2 サーバーへ接続する

最も基本的な形は次である。

リスト A.1 SSH 接続の基本

```
$ ssh user@server.example.jp
```

user はサーバー上の自分のユーザー名、server.example.jp はホスト名である。ポート番号が既定の 22 でないなら -p を付ける。

リスト A.2 ポート番号を指定する

```
$ ssh -p 2222 user@server.example.jp
```

接続後は、ローカルのターミナルと似た形でコマンドを打てる。ただし、そこで見えているファイルは自分の手元ではなく、サーバー側のファイルである。この区別が曖昧になると事故の原因になるので、hostname、whoami、pwd を見て今どこにいるかを確かめる習慣を持った方がよい。

A.3 SSH 鍵を作る

毎回パスワードを打つ代わりに、SSH 鍵を使うことが多い。鍵は公開鍵と秘密鍵の 2 つ 1 組であり、公開鍵をサーバーへ登録し、秘密鍵は自分だけが持つ。

リスト A.3 SSH 鍵を作る

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

既定値のまま進めると、多くの場合は ~/.ssh/id_ed25519 が秘密鍵、~/.ssh/id_ed25519.pub が公開鍵になる。 .pub が付いている方だけを相手へ渡す。 .pub が付いていない方は秘密鍵なので、決して共有してはいけない。

リスト A.4 鍵ファイルを確認する

```
$ ls -l ~/.ssh
```

秘密鍵の権限が緩いと SSH が拒否することがある。その場合は次のように直す。

リスト A.5 権限を整える

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

A.4 公開鍵をサーバーへ登録する

公開鍵をサーバーへ登録する最も簡単な方法は `ssh-copy-id` である。

リスト A.6 `ssh-copy-id` の基本

```
$ ssh-copy-id user@server.example.jp
```

特定の公開鍵を使いたいなら `-i` で指定できる。

リスト A.7 使う公開鍵を明示する

```
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server.example.jp
```

`ssh-copy-id` は、指定した公開鍵をサーバー側の `~/.ssh/authorized_keys` へ追加する。つまり、「手元の公開鍵を安全に送り、向こうで追記する」という面倒な作業を代わりにやってくれる。

A.4.1 `ssh-copy-id` が使えない場合

`ssh-copy-id` が使えない環境では、公開鍵の中身を手で登録する。

リスト A.8 公開鍵の中身を表示する

```
$ cat ~/.ssh/id_ed25519.pub
```

表示された 1 行全体を、サーバー側の `~/.ssh/authorized_keys` へ追記する。サーバーへ通常のパスワードログインができるなら、自分でログインして追記してもよい。

A.5 `ssh-agent` を使う

パスフレーズ付きの鍵を使う場合は、`ssh-agent` に読み込ませておくと毎回の入力を減らせる。

リスト A.9 `ssh-agent` を使う

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
```

A.6 ~/.ssh/config を書く

接続先が増えると、毎回長いホスト名やポート番号や鍵ファイル名を書くのは面倒になる。そこで ~/.ssh/config を使う。

リスト A.10 SSH 設定ファイルの例

```
Host labserver
  HostName server.example.jp
  User ikomae
  Port 2222
  IdentityFile ~/.ssh/id_ed25519
```

これを書いておけば、以後は次のように短く書ける。

リスト A.11 別名で接続する

```
$ ssh labserver
```

config ファイル自体も秘密情報に近いので、`chmod 600 ~/.ssh/config` としておく方が安全である。

A.7 GitHub に SSH 鍵を登録する

通常のサーバーとは違って、GitHub には `ssh-copy-id` を使わない。GitHub はシェル接続先ではなく、アカウント設定に公開鍵を登録する相手だからである。

最も確実なのは、GitHub の設定画面の SSH and GPG keys へ公開鍵を登録する方法である。もし GitHub CLI `gh` を使うなら、認証時に SSH 鍵の作成や登録を案内させることもできる。

リスト A.12 GitHub CLI で SSH を選ぶ例

```
$ gh auth login --git-protocol ssh --web
```

GitHub CLI を使う場合は、既存の鍵を検出してアップロードするか、新しい鍵を作るかを対話的に選べる。GitHub CLI を使わない場合は、`cat ~/.ssh/id_ed25519.pub` で表示した公開鍵を Web 画面へ貼り付ければよい。

A.8 ファイル転送は rsync を基本にする

小さなファイルを 1 回だけ送るなら `scp` でもよいが、解析ではディレクトリ全体を何度も同期することが多い。その場合は `rsync` を使う方が自然である。

A.8.1 scp の最小例

リスト A.13 scp の例

```
$ scp result.csv user@server.example.jp:~/work/  
$ scp user@server.example.jp:~/work/result.csv .
```

1 行目は手元からサーバーへ送り、2 行目はサーバーから手元へ持ってくる。ディレクトリごと送りたい場合は `-r` を付ける。

A.8.2 rsync を推奨する理由

`rsync` は、差分だけを転送し、途中で止まっても再開しやすく、除外パターンも柔軟に書ける。そのため、解析用ディレクトリやコード一式をサーバーへ送る用途では `scp` より向いている。

リスト A.14 rsync の基本例

```
$ rsync -aP ./analysis/ user@server.example.jp:~/work/analysis/
```

よく使うオプションの意味は、次のように 1 つずつ読んだ方が理解しやすい。

- `-a` archive モードである。再帰コピーを行い、時刻や権限などもまとめて保つ。
- `-P` `--partial` `--progress` の省略形である。途中ファイルを残しつつ進捗を表示する。
- `-v` 何を転送しているか詳しく表示する。
- `-z` 転送時に圧縮する。低速回線では有利だが、既に圧縮済みの大きなデータでは効果が薄いこともある。
- `--exclude` 特定のファイルやディレクトリを転送対象から除外する。

リスト A.15 除外指定付きの例

```
$ rsync -aPv \  
--exclude='.venv/' \  
--exclude='__pycache__/' \  
--exclude='build/' \  
--exclude='.mypy_cache/' \  
./analysis/ user@server.example.jp:~/work/analysis/
```

仮想環境、キャッシュ、ビルド成果物はサーバー側で作り直せることが多いので、転送から外した方が軽くなる。

A.8.3 末尾のスラッシュに注意する

`rsync` では、送信元パスの末尾に `/` を付けるかどうかで意味が変わる。

リスト A.16 末尾スラッシュの違い

```
$ rsync -aP analysis user@server:~/work/  
$ rsync -aP analysis/ user@server:~/work/analysis/
```

1 行目は `analysis` というディレクトリごと送る。2 行目は `analysis` の中身を送る。この違いは非常に重要であり、意図しない階層を作る原因になりやすい。

付録 B Git と GitHub の基本

B.1 Git と GitHub は別物である

Git は、手元のファイルの変更履歴を記録するためのソフトウェアである。これに対して GitHub は、Git リポジトリを置いて共有するためのサービスである。

- Git: ローカルマシンでも単独で使える版管理システム
- GitHub: Git リポジトリをホストし、共有、Issue、Pull Request などを提供するサービス

したがって、`git init`、`git add`、`git commit` は Git の話であり、`git push` で GitHub へ送る段階になって初めて GitHub が関係してくる。

B.2 リポジトリと公開範囲

リポジトリは、コード、データ、履歴をまとめて管理する単位である。GitHub へ置く場合は、公開範囲も意識する必要がある。

- `public`: 誰でも見られる
- `private`: 明示的に許可した人だけが見られる
- `internal`: 組織内だけで見られる。主に GitHub Enterprise の文脈で使う

授業用の配布物、研究中のコード、機密データを含むリポジトリでは、公開範囲の設定を誤ると問題になる。GitHub で新しいリポジトリを作るときは、まず `visibility` を確認した方がよい。

B.3 Git で何をしているのか

Git は、ファイルを 1 回で全部自動保存してくれるわけではない。ふつうは次の 3 段階で進む。

1. 作業中のファイルを編集する
2. 記録したい変更を `git add` で選ぶ
3. `git commit` で履歴として確定する

この 2 段階目の「いったん選ぶ場所」をステージングエリアという。`git add` と `git`

commit の違いを理解するには、ここが重要である。

B.4 最初の設定

最初に、自分の名前とメールアドレスを設定しておく。

リスト B.1 Git の初期設定

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your_email@example.com"
```

これをしておかないと、コミット時に誰の変更なのかが分からなくなる。

B.5 新しいリポジトリを作る

まだ Git 管理されていないディレクトリなら、git init で始める。

リスト B.2 新しいリポジトリを作る

```
$ mkdir analysis_work
$ cd analysis_work
$ git init
$ git status
```

git status は最重要コマンドである。今どのファイルが未追跡か、どれが変更済みか、何がコミット待ちかを確認できる。迷ったらまず git status を見る、という習慣を付けるとよい。

B.6 編集、確認、記録の流れ

日常的な流れは、編集して、差分を確認して、必要なものだけをコミットする、である。

リスト B.3 ローカル作業の基本

```
$ git status
$ git diff
$ git add analyze.py
$ git diff --cached
$ git commit -m "Add first analysis script"
$ git log --oneline
```

git diff は、まだ add していない変更を見る。git diff --cached は、add 済みで次のコミットへ入る変更を見る。git log --oneline は、これまでのコミット履歴を短く確認する。

B.6.1 git status の読み方

git status は、少なくとも次の 3 種類を見分けられるようになるとよい。

- untracked: まだ Git が追跡していない新規ファイル

- modified: 追跡中だが、まだ add していない変更
- changes to be committed: add 済みで、次のコミットへ入る変更

つまり、git status は「いま何がどの段階にあるか」を示す画面である。

B.6.2 git add は何をしているのか

git add は「変更を次のコミット候補として選ぶ」コマンドである。保存ではない。したがって、git add した後でも、コミット前にさらに編集すれば、その追加分はまだコミット対象へ入っていない。

リスト B.4 複数段階で確認する例

```
$ git diff
$ git add analyze.py
$ git diff --cached
```

この 2 回の diff を見分けられるようになると、Git の挙動がかなり分かりやすくなる。

B.7 .gitignore を使って除外する

生成物まで毎回コミットすると、履歴が見にくくなる。Python 解析なら、仮想環境、キャッシュ、ビルド成果物などは除外することが多い。

B.7.1 .gitignore を書く

.gitignore は、Git に「このパターンのファイルは追跡しなくてよい」と教えるためのファイルである。リポジトリのルートに置くことが多い。

リスト B.5 .gitignore の例

```
.venv/
__pycache__/
*.pyc
build/
dist/
.pytest_cache/
```

このファイルを作ると、まだ追跡されていない .venv/ や build/ は git status に出にくくなり、誤ってコミットしにくくなる。

B.7.2 すでに追跡済みなら git rm --cached が必要

.gitignore は「これから追跡しない」ための設定である。すでに一度コミットしたファイルには、そのままでは効かない。

リスト B.6 追跡済みの生成物を外す

```
$ git rm --cached -r .venv __pycache__ build
$ git commit -m "Stop tracking generated files"
```

`git rm --cached` は、作業ディレクトリの実ファイルを消さずに、インデックスからだけ外す。そのため、「手元には残したまま Git の追跡対象から外す」という目的に向いている。

B.7.3 uv プロジェクトで何をコミットするか

uv を使うプロジェクトでは、通常 `.venv/` はコミットしない。一方で、`pyproject.toml`、`uv.lock`、`.gitignore`、そしてプロジェクト単位で版を固定したいなら `.python-version` はコミット候補になる。

B.8 既存のリポジトリを持ってくる

他人のプロジェクトや GitHub 上の教材は、`git clone` で手元へコピーする。

リスト B.7 リポジトリを複製する

```
$ git clone https://github.com/example/project.git
$ cd project
$ git remote -v
```

`git clone` は、履歴も含めて手元へ持ってくる。`git remote -v` は、どの URL が接続先として設定されているかを確認する。

B.9 手元のリポジトリを GitHub へ初めて載せる

ここまでは「GitHub 上にあるものを取得する」話だったが、逆に、手元で `git init` して育てたプロジェクトを自分の GitHub へ載せたい場面も多い。その場合は、まず GitHub 側で空のリポジトリを作り、次に手元のリポジトリから接続先を登録して `push` する。

B.9.1 GitHub 側で空のリポジトリを作る

GitHub の Web 画面で `New repository` を開き、リポジトリ名と公開範囲を決める。このとき、手元にすでに履歴があるなら、最初は空のリポジトリとして作る方が簡単である。つまり、`Add a README file`、`Add .gitignore`、`Choose a license` には最初は触れない方がよい。

最初から `README` や `.gitignore` を GitHub 側で作ると、リモート側に 1 個目のコミットができる。その状態で手元の履歴をそのまま `push` すると、履歴の始点が一致しないため、初心者には扱いにくい状況になりやすい。したがって、既存のローカルリポジトリを載せるときは「空の GitHub リポジトリ」を作る、という方針で覚えた方が安全である。

B.9.2 README や LICENSE を先に作っていた場合

すでに GitHub 側で `README` や `LICENSE` を作ってしまったなら、リモート側には先にコミットが 1 つ以上存在する。その場合は、いきなり `git push` するのではなく、まずリモート側の履歴を取り込んでから `push` しなければならない。

リスト B.8 GitHub 側に初期コミットがある場合

```
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git pull origin main --allow-unrelated-histories
$ git push -u origin main
```

`--allow-unrelated-histories` が必要なのは、手元のリポジトリと GitHub 側のリポジトリが別々に作られており、履歴の出発点が一致していないからである。この `pull` によって、GitHub 側の README や LICENSE のコミットを自分の履歴へ取り込んでから、その続きとして `push` できるようになる。

もし手元にも README や LICENSE があり、同じ名前のファイルを両側で作っていたなら、`git pull` の途中で競合が起こることがある。その場合は、どちらの内容を残すかを自分で決めて修正し、`git add` と `git commit` で競合解消を確定してから、あらためて `git push -u origin main` を実行する。

B.9.3 接続先を登録して最初の push を行う

GitHub が表示する URL を確認し、手元のリポジトリで `origin` として登録する。多くの場合、次の流れで足りる。

リスト B.9 既存のローカルリポジトリを GitHub へ載せる

```
$ git status
$ git branch --show-current
$ git branch -M main
$ git remote add origin git@github.com:owner/project.git
$ git remote -v
$ git push -u origin main
```

`git branch --show-current` は、いま自分がどのブランチにいるかを確認する。`git branch -M main` は、既定ブランチ名を `main` へそろえるための書き方である。`main` ではなく既存のブランチ名をそのまま使いたいなら、その名前でも `push` してよい。

`git remote add origin ...` は、GitHub の URL を `origin` という別名で登録する。`git remote -v` を見れば、正しい URL が設定されたかを確認できる。`git push -u origin main` の `-u` は、以後このローカルブランチが `origin/main` を追跡する、という設定である。初回にこれを付けておけば、次回以降は `git push` や `git pull` を短く書きやすい。

B.9.4 origin がすでにある場合

すでに `origin` が登録されている状態で `git remote add origin ...` を実行すると、エラーになる。その場合は、いま何が設定されているかを確認してから、必要なら URL を差し替える。

リスト B.10 origin を確認し、必要なら差し替える

```
$ git remote -v
$ git remote set-url origin git@github.com:owner/project.git
```

```
$ git remote -v
```

教材を別のリポジトリから複製して流用した場合や、HTTPS から SSH へ切り替えたい場合に、この操作が必要になることがある。

B.10 GitHub と同期する

ローカルでコミットしただけでは、まだ GitHub には反映されない。GitHub 側から最新を取り込むのが pull、手元のコミットを送るのが push である。

リスト B.11 基本的な同期

```
$ git pull --ff-only  
$ git push origin main
```

git pull --ff-only は、自分がまだ追加のコミットを持っていないときだけ安全に進める書き方である。履歴が分岐している場合は止まるので、意図しないマージを避けやすい。

B.11 GitHub で使う URL には HTTPS と SSH がある

GitHub のリポジトリ URL には、少なくとも HTTPS と SSH の 2 通りがある。

リスト B.12 HTTPS と SSH の例

```
https://github.com/owner/repo.git  
git@github.com:owner/repo.git
```

HTTPS は見慣れた URL で扱いやすい。一方 SSH は、公開鍵認証を済ませておけばパスワードやトークン入力を減らしやすい。どちらを使ってもよいが、混在させると混乱しやすいので、最初に方針を決めた方がよい。

B.12 GitHub の認証: PAT と SSH 鍵

GitHub では、HTTPS 経由の Git 操作で通常のアカウントパスワードは使えない。現在は、個人用アクセストークン (PAT) を使うか、SSH 鍵を登録して使うのが基本である。

B.12.1 PAT を使う

HTTPS を使う場合、プライベートリポジトリへの push や pull では PAT を使う。

1. GitHub の Settings を開く
2. Developer settings から Personal access tokens へ進む
3. fine-grained token か classic token を作る
4. 対象リポジトリと必要最小限の権限を選ぶ
5. 発行したトークンを安全に保管する

認証時には、ユーザー名は GitHub のアカウント名、パスワード欄には通常のパスワードではなく PAT を入れる。組織アカウントでは、SAML SSO の承認が別に必要なこともある。その場合は、発行したトークンをその組織用に追加承認しないと使えない。

B.12.2 SSH 鍵を使う

SSH を使う場合は、付録 A で作った公開鍵を GitHub の SSH and GPG keys へ登録する。GitHub CLI `gh auth login --git-protocol ssh --web` を使うと、その作業を対話的に進めやすい。

B.13 初心者が押さえるべき最小ワークフロー

最初のうちは、次の順序だけ守れば十分である。

1. `git status` で状況を見る
2. `git diff` で変更内容を読む
3. `git add` で記録したいファイルを選ぶ
4. `git diff --cached` でコミット内容を再確認する
5. `git commit -m "..."` で記録する
6. GitHub とやり取りするなら `git pull --ff-only` と `git push` を使う

この流れを身に付ければ、教材の取得、uv プロジェクトの再現、共同研究でのコード共有まで一通り対応できる。最初からブランチ運用や複雑な履歴操作まで覚える必要はないが、`status` と `diff` を読まずに `add` や `commit` をする癖だけは避けた方がよい。